UCLouvain

epl

École polytechnique de Louvain

Profiling Novice Programmers' Behavior through the Analysis of Incremental Programming Submissions

Author: **François Junior Meli Ngueunkeung**
Supervisors: **Kim Mens, Siegfried Nijssen**
Co-supervisor: **Guillaume Steveny**
Reader: **Olivier Goletti**
Academic year 2025–2026
Master [120] in Computer Science and Engineering

Abstract

When students learn to program with an automatic grading platform, every attempt they make is recorded. What the course keeps is the final score. How a student reached that score, the order of the attempts, the size of each edit and the time between them, is also recorded but typically left unused. This thesis asks what we can learn from that once the score is set aside, and whether the way a student works on the weekly coursework is related to the way the same student works in an exam.

The data comes from an introductory Python course at UCLouvain and from its INGInious *autograder*, the platform that runs each submission against hidden tests and returns a score. We study the **569 students** who appear in both settings: the weekly coursework they did across the term, and the January exam, taken under a locked-down browser where a submission reports only whether the code runs. We describe each student by behavior features alone, with the grade held out, and group them with Ward hierarchical clustering, choosing the number of groups from the data rather than fixing it in advance. We then read the same data at a finer level, sorting each sequence of submissions to one question into one of six shapes defined only by what the sequence shows, and following individual students through their code from one submission to the next.

The main result is that the submission record separates students the grade cannot. Grouping on behavior alone gives four ways of working in the exam and three in the coursework, each with its own pattern. Those groups differ in what students did, not only in what they scored: two students who end on the same mark can have worked in opposite ways, and a low mark in particular turns out to cover several quite different ways of failing. The second result is that a way of working belongs to the setting it was seen in, the coursework or the exam, rather than carrying from one to the other. Far more exam sequences than coursework sequences never reach the pass mark, and the two groupings are only weakly related, so knowing how a student worked in the coursework says little about how they worked in the exam.

Three limits come with these results. The groups are not sharply separated, so they describe tendencies rather than distinct kinds of student. The study covers one course at one institution, so these particular groups belong to this course, and another course would not be expected to produce the same ones. And a submission record shows what a student did, not why, so no claim is made here about intent, motivation or understanding.

Keywords: novice programming, programming process data, educational data mining, hierarchical clustering, submission sequences, introductory computing education.

Acknowledgments

I would like to thank the supervisors of this thesis: Prof. Kim Mens and Prof. Siegfried Nijssen for their guidance throughout this thesis. Across weekly meetings, continuous support and feedback, Prof. Mens provided invaluable advice on research design and methodology, while Prof. Nijssen contributed his expertise in AI and machine learning.
I also want to thank Guillaume Steveny who offered valuable perspective and overall advice alongside Prof. Kim Mens and Prof. Siegfried Nijssen.
I am also grateful to Dr. Olivier Goletti for his advice during the Poster session presentation, which proved helpful for the qualitative analysis of this master thesis.
Special thanks to Julien Liénard for providing the first versions of the INGInious data extraction, which laid the foundation for this thesis.
I thank the students who completed the survey for the time they gave to it.
Furthermore, I am deeply grateful to my family, partner, and friends for their encouragement, support, and confidence throughout this work.
Finally, this thesis reflects the belief that understanding how novice programmers think and learn can benefit computer science education.

Disclaimer

This thesis uses the Python scientific stack for data processing and figures, and historical submission data from the INGInious platform used by the course.
Generative AI tools were used to assist with data-processing code and with drafting parts of the manuscript. All results come from the scripts described in Appendix A and can be regenerated from them.
The research questions, the choice of features and thresholds, the decisions about which data to keep, the reading of the groups and of the individual cases, and the conclusions are mine.
The works discussed in Chapter 2 were read before being cited. The statistical references cited only for the definition of a measure were not read in full.

Contents

Chapter 1

Introduction

Learning to program is learning by doing. A student writes a piece of code, runs it, sees it fail, changes it, and runs it again, until it works or until they move on. Each of these steps is a small decision, and their order tells a story: a student who reaches a working answer in two tries has probably not worked in the same way as one who submits twenty times, even if both end with the same code. How a student moves through these attempts is part of how they learn.

Very little of this is normally kept. Once a task is graded, a student's work becomes a single number, and the attempts that led to it are forgotten. The teacher is left with the outcome and not the path. In a small group a tutor can watch students work and fill in the rest; in a first-year course with hundreds of students that is no longer possible, and the process disappears behind the final mark.

The process is not truly lost, though. A programming course that uses an autograder, like the one we study, usually keeps every submission a student makes, together with its code, its time, and its score. The information needed to see how students work is already there; it is simply not read. This thesis reads it. Working from the submission histories of the first-year introductory Python course at UCLouvain, where students submit their code to the INGInious autograder (both described in Chapter 4), we describe how students go about their programming, and we group students who go about it in similar ways.

1.1 Problem

A student's work on a task is, in the end, recorded as one score. That score says whether the student succeeded, but not how they got there: whether they found the answer quickly or after a long series of attempts, whether they changed a few lines at a time or rewrote everything at once, whether they took long breaks between attempts or worked straight through. Two students with the same mark can have

worked in opposite ways, and a teacher who sees only the mark cannot tell them apart. With a large class this is the normal situation: the data that would show how students worked exists, but there is no simple way to read it. The problem this thesis addresses is how to turn those raw submission histories into a description of student behavior that a teacher can actually read.

1.2 Objective

Our aim is to describe student programming behavior from submission histories. For each student we compute a set of behavior features from their history: how many tasks they attempt, how many attempts they make on each, how quickly they resubmit, how long they pause, how much their code changes between attempts, how often an attempt raises their score, and a few properties of the code they finally hand in. We then group students whose features are alike, so that each group stands for one recognizable way of working. A mark measures success rather than behavior, so we do not use it to form the groups; it serves only as a later point of comparison. We do this in two settings, the weekly coursework across the term and the January exam, which lets us ask whether a student works the same way when practicing and when sitting an exam. The study stays descriptive throughout: we report what the submissions show, and we are careful not to read intentions we cannot check.

1.3 Research questions

The thesis is organized around three questions.
RQ1. Can students be grouped into distinct, recognizable ways of working, using only their submission histories? If such groups exist and hold up, they give teachers a way to talk about how students work that does not reduce to a grade. We build these groups and test how stable they are in Chapter 5.
RQ2. Do these groups differ in how students actually work through a task, beyond the score they reach? A behavior group is only worth having if it tells us more than the grade already does. We check this by reading individual submission sequences and a set of case studies in Chapter 6.
RQ3. Is the way a student works during the year related to the way they work in the exam? If year behavior carried over to the exam, early coursework could hint at how a student will later work under pressure; if it does not, each setting has to be read on its own. We compare the two groupings in Chapters 5 and 6.

1.4 Contributions

This thesis makes four contributions, one methodological and three empirical.

The first is a reproducible analysis pipeline. It takes the raw submission logs and turns them, step by step, into behavior features, student groups, and the figures shown in this document, with each step written as a separate script whose output can be inspected on its own. This is what lets every number in the thesis be traced back to the code that produced it; the pipeline is described in Chapter 4 and Appendix A.

The second is a set of behavior groups for both settings, each with its own recognizable signature, presented in Chapter 5. Since the groups are formed from behavior alone, we can then ask what they tell us that the grade does not.

The third is a reading of behavior at the level of the single attempt, developed in Chapter 6: a small vocabulary of episode shapes and a handful of case studies of real students, which show how two students in the same group, or even with the same score, can work very differently.

The fourth is a comparison of the same students across the year and the exam, made at the level of the groups in Chapter 5 and student by student in Chapter 6. We find the two to be related only loosely: how a student worked during the year says little about how they work in the exam. We report this as an observation about these students, not as a general rule.

1.5 Thesis outline

The rest of the document follows the path from data to interpretation. Chapter 2 reviews the research this work builds on: what is known about how many students pass an introductory course, how earlier studies have read the way novices program from the process rather than from the result, what has been measured from submission logs, and how students have been grouped from such data. Those studies answer how novices work inside a single setting; none of the work reviewed there compares the same students across ordinary coursework and a real exam, which is the question this thesis adds. Chapter 3 defines the methods used later: hierarchical clustering with Ward linkage, the four measures used to choose the number of groups, and the measures used to compare two groupings of the same students. Chapter 4 presents the course and the INGInious platform in more detail, the two datasets (the year's coursework and the January exam), and the cleaning steps that decide which students and submissions we keep. Chapter 5 is the quantitative core: it defines the behavior features, forms and validates the groups, and compares the two groupings, answering RQ1 and the first half of RQ3. Chapter 6 goes down to individual submission sequences, defines the *episode shapes*,

the small set of patterns that a single run of submissions to one question can follow, and works through the case studies that answer RQ2; it also follows the same students across the two settings, which completes the answer to RQ3. Chapter 7 brings the answers together, states the threats to validity, and points to further work. The appendices collect the reproducibility details, additional figures and tables, and the case-study code.

Chapter 2

Related Work

This chapter reviews the work this thesis builds on. We look at four things in turn: what is actually known about how many students pass an introductory programming course, how researchers have studied student behavior from the working process rather than from the finished program, what has been measured from submission logs, and how students have been grouped from such data. For each work we state what was done, what was found, and where the authors or we see a limit.

2.1 Pass rates in introductory programming

A common way to open a thesis on novice programmers is to state that failure rates are high. That claim is worth checking before it is used. Watson and Li [26] reviewed the reported outcomes of 161 introductory courses, run between 1979 and 2013 at 51 institutions in 15 countries. They report a mean pass rate of 67.7%, with a 95% confidence interval of 65.3% to 70.1%. Two of their results matter more than the mean. First, individual course pass rates run from 23.1% to 96%, so the mean says little about any single course. Second, they found no statistically significant change over the whole period ($F(21, 139) = 0.486$, $p = 0.971$).

Their own reading is cautious. They write that 67.7% "is not an alarmingly low pass rate", and they state that they cannot say why the remaining students did not pass: only 36 of the 161 course reports separated a failure from a withdrawal. Their stated limits are the uneven definition of a pass across their sources, the possibility of publication bias toward better outcomes, and coverage of only 15 countries. We add one more. They average the pass rate of 161 courses, and every course counts the same whether it had 20 students or 2,000. So 67.7% is the pass rate of the average course, not the share of students who pass.

What can be taken from this is narrower than the usual claim. A sizeable

minority of students do not pass, the spread between courses is wide, and the published record does not say why.

Student and teacher opinions have also been collected. Lahtinen, Ala-Mutka, and Järvinen [15] surveyed 559 students and 34 teachers across six universities. The items rated hardest by students were not language concepts but parts of the working process: finding bugs in their own program (mean 3.28 on a five-point scale), designing a program for a task (3.12), and dividing functionality into procedures (3.10). Teachers rated almost every item harder than the students did. The authors state plainly that "the responses are subjective opinions of the people who answered" and that students "do not always see their difficulties". Nothing in that study measures what students actually did, so it reports perception and not behavior. That gap between what students report and what they do is one reason to work from recorded submissions.

2.2 Novice behavior as a process

The idea that the final program hides how it was produced is old. Robins, Rountree, and Rountree [22], reviewing the psychological and educational literature on programming, report that while strategy use shapes the finished program, the strategies themselves "cannot (in most cases) be deduced from the final form of the program". Their review is narrative, with no stated search method or list of included papers, and they mark their own central proposal, the distinction between effective and ineffective novices, as speculative. It is a source for framing rather than for evidence. Their point about the finished program is nevertheless the reason process data is collected at all.

The earliest behavioral vocabulary in this area comes from clinical observation. Perkins et al. [20] sat with school pupils learning BASIC and LOGO, about 30 and 25 students respectively, gave them a programming problem, and probed them when they were stuck. They describe *stoppers*, who meet a difficulty, feel at a complete loss and become unwilling to explore the problem any further, and *movers*, who try one idea after another, along with *tinkering*, a pattern of small repairs and retests that they place inside the mover behavior rather than beside it. The authors are explicit that the article is not a technical report and that their conclusions are tentative. The sample is small, the setting is a 1980s school, and no counts or statistics are given. Two further points matter for us. Their labels name what the student is taken to be doing, and this thesis avoids such labels, since intent cannot be read from a log. And the thing they say matters most is whether a student looks carefully at their own code and works out what it really does. That happens in the student's head, between two submissions, so nothing in our data shows whether a student did it.

Jadud [12] moved the same idea to recorded data. Working with first-year Java students in the BlueJ environment, he collected a snapshot at every compilation and built the Error Quotient, a number computed from consecutive pairs of compilation events that describes how much a student repeats the same syntax error. The reported relationship with attainment is weak, and he says so: the fit against average assignment grades is $R^2 = 0.11$ and against the final exam $R^2 = 0.25$, which he calls "very poor". Two limits are worth carrying forward. The parameters of the measure were chosen by a search that maximized the spread between students, and snapshots were recorded only in the public laboratories, so work done elsewhere is absent. Filtering to students with at least three recorded sessions removed 40% of the sample.

A passing result does not show whether a student understood their own code. Kennedy and Kraemer [14] asked ten introductory students to think aloud while rebuilding a small program in C from a provided executable, with no starter code and no automatic grading. Seven of the ten reproduced the required behavior. The authors observed that this did not tell them whether the students had understood what they wrote: some finished with, in their words, "erroneous code that coincidentally worked", reached by trial and error and by edits made to satisfy compiler messages. One student added a piece of code he said he was "99% certain" was not needed, only because the compiler complained without it. He then checked that the program ran, and moved on. Their headline observation is that students "showed uncertainty regardless of success at task completion". This is ten students, one task and one hour, so it is an observation and not a general law.

Henry [9] reaches a similar boundary from a different direction. Her thesis builds concept inventories, multiple-choice instruments whose wrong options are drawn from documented misconceptions, and administers them before and after teaching in a first-year Python course. She states that a student who passes a summative assessment has learned something, but that passing it does not guarantee the quality of that learning. In her data, students who reported prior programming instruction still failed the diagnostic given before the theory class, and the confusion between a mathematical variable and a programming variable was still present after two years of courses. She is careful about her own instruments, noting that they still require validation and that her profiles mainly show the effect of the teaching actions.

Self-regulated learning is often cited near this material. We mention it only to set a boundary. Zimmerman [27] describes learning as a cycle of forethought, performance and self-reflection, driven by goals, strategies and beliefs. That article is a conceptual overview rather than a study, it contains no log data, and he notes that many of the processes involved are "generally covert". Our data cannot reach them, so this thesis makes no claim about self-regulation.

2.3 Measures built from submission logs

The clearest picture of research on students' programming processes comes from Ihantola et al. [11], a report written jointly by 16 researchers as a working group at the ITiCSE conference, which reviews 76 papers published between 2005 and 2015. Three of their counts frame our own work. 62 of the 76 studies (81%) were conducted at a single institution. Only 2 studies (3%) covered between 500 and 1000 students. And they warn researchers "to refrain from drawing strong conclusions from such studies, especially if the data comes from a single context or course".

They also point out that this kind of data can be recorded in more or less detail. The most detailed recordings include every keystroke a student types. The least detailed keep only the files the student submitted, which is what we have. Of systems like ours they write that "a significant limitation is the relative sparseness of these events, and the lack of visibility into solution states or actions between submission events": the submissions are far apart, and nothing a student does between two of them is recorded. That limitation applies directly to our data.

Their most useful result for us is a reproduction. Some of those same 16 authors recomputed Jadud's Error Quotient on a Python course with 48,037 submissions from 476 students, and found that the resulting scores "were correlated neither with assignment nor exam marks at the $p < 0.05$ significant level". Their conclusion is that "the details of the context contribute to the outcomes". A second case study failed to replicate an earlier finding, and the cause they traced was a detail absent from the original paper: the starter templates of one system always compile and those of the other do not. Both cases argue for describing a pipeline in full and for treating a behavioral result as tied to its setting.

Several studies define a unit inside the submission stream. Liu and Paquette [16] analyzed autograded Java submissions from 756 students in one semester and 442 in another, and defined a *debugging episode* as "a sequence where a student receives the same test error in multiple submissions in a row". Their descriptive results show a narrow repertoire: deletions and small edits appear in more than 40% of submissions, while print statements appear in only 17% and 18% of episodes across the two semesters. They also report that small edits are associated with the more efficient episodes, which runs against the older reading of small changes as aimless. Their models explain little, with a pseudo R^2 near 0.12, and they state that their frequency features "are not sufficient to discover the latent patterns of successful debugging processes".

Oliveira et al. [19] studied five semesters of a CS2 course through commit histories, and defined struggle from the data rather than in advance: they observed that a failing teaching-staff test was normally fixed within four commits, so a test failing for four or more consecutive commits marks a struggling state. They

deliberately do not use grades, both because their course shows a ceiling effect and because, in their words, high grades might not mean learning. Their findings include that students slow down and narrow their edits while stuck, and that help-seeking did not rise during those stretches. Their setting is much richer than ours, since they also observe student-written tests, coverage, office hours and forum posts.

Two studies model the order of events, not only how many of each there were. Lokkila [17] treats each student's submission history as a path through eight states and counts how often that student moves from each state to each other one, giving one table of transitions per student; students are then grouped from those tables, using 544,835 submissions from 1,174 students. Ardimento et al. [2] records what students do inside their editor and builds, for each student, a diagram of the steps they went through and in what order, from six students and 427 sessions. Both are alternatives to the approach taken here, and both name their own limits: Lokkila states that k-means was the only method he tried for grouping the students, and that a qualitative analysis is needed to check the groups it produced, while Ardimento and colleagues write that "global generalization is not possible" from six students. A diagram of that kind needs every recorded step to carry a label saying what it was. Our records hold submissions with timestamps and grading outcomes rather than editor commands, so building one would require a labeling step that we did not perform.

2.4 Grouping students

Grouping students is a recognized task in educational data mining rather than a novel one. Romero and Ventura [23] organize educational data mining by educational task and give "Grouping students" its own category, listing hierarchical clustering beside k-means and model-based methods without arguing for any of them. Baker and Inventado [3] place clustering inside *structure discovery*, which they define as finding structure "without any ground truth or a priori idea of what should be found", in contrast with prediction, where labels must exist first. They also name a choice this thesis has to make, what to group over: clusters can be built over students, "to investigate similarities and differences among students", or over "student actions ... to investigate patterns of behavior". Neither source offers guidance on choosing the number of groups or on validating a grouping, and Baker and Inventado note that structure discovery generally receives less attention to validation than prediction does.

The closest existing work is the thesis of Fraser [8], who profiled students using his own feedback platform over four semesters. His design matches ours in shape. He profiles two different units: each single work session, and each student across a

whole semester. He groups both with k-means, then tests with a chi-square how the session profiles map onto the semester profiles. He reports seven semester profiles and six session profiles, with plain descriptive names such as "persistent struggler", "persistent progressor" and "near non-participant". Most students do not stay in one session profile: their sessions fall mainly into two of them, and between 40 and 54% of sessions are of a different profile from the session before. Two of his results are relevant to what we should expect. The clustering quality was low, with what he calls a low average silhouette score across the seven profiles, and the link between session profiles and semester profiles was real but loose: some session profiles never occurred under some semester profiles, and some semester profiles matched no session profile at all.

His stated limits are what separate his setting from ours. Participation was voluntary and he reports a strong bias toward higher-performing students; his platform was not the only tool available, so it records discrete points rather than the whole process; and submissions were not reliably tied to a course or an assignment, which he says limited the analysis. In the setting studied here the platform is the course's own assessment system, participation is not optional, and every submission belongs to a known question.

Profiling has also been done without process data at all. Altun and Mazman [1] built profiles of 100 undergraduate students from measured and self-reported characteristics: spatial orientation, mental rotation, visual and verbal working memory, mathematics grades and programming self-efficacy. Their groups were not found in the data. They split the students in two, those scoring above the class average and those scoring below, and then described each side, with no test of whether any variable actually differs between the two. The contrast with the present work is one of data and of direction: their profiles describe what students are and report, formed after splitting on the result, while ours describe what students did, formed with the result held aside.

At the other end sits prediction. Piech et al. [21] model sequences of exercise outcomes with recurrent neural networks and predict whether the next answer will be correct, reaching an area under the curve of 0.86 against 0.69 for the best previously reported result on the same benchmark. The model is also shown to recover structure between exercises without expert labels. Two properties keep it away from the questions asked here. Its input is a sequence of exercise identifiers with a correct or incorrect flag, so the timing and the code that this thesis uses would be discarded, and what it describes is the exercise rather than the student. The authors also state the practical boundary themselves: such models "require large amounts of training data, and so are well suited to an online education environment, but not a small classroom environment". Their datasets contain 15,931 and 47,495 students.

2.5 Related work conclusion

Three things follow from this review.
First, the finished program and the final grade are poor witnesses of how the work was done. This is stated in the review literature [22], shown in observation [20, 14], and acted on by the studies that build measures from submission sequences [12, 16, 19]. Several of those authors avoid grades on purpose.

Second, results in this area are modest and tied to their setting. The Error Quotient explains 11% and 25% of the variance in two attainment measures [12], and lost its relationship with marks entirely when recomputed in another course [11]. Clustering quality was low in the closest comparable study [8], and models of debugging behavior explain little [16]. A weak result is the normal outcome here, and reporting one is not a failure of the analysis.

Third, and this is where the present work differs, none of the studies above compare how the same students work in ordinary coursework and in a real exam. Fraser compares sessions with semesters on one voluntary platform, Lokkila and Liu and Paquette stay inside course assignments, Oliveira and colleagues stay inside projects, and the surveys describe no such study. The taxonomy of Romero and Ventura [23] has no category for comparing two groupings of the same students in two settings. The exam studied here is taken under supervision in a locked-down browser, where a submission reports only whether the code runs, so it is a setting the coursework cannot imitate.

This thesis makes that comparison. It groups students by behavior with the grade held aside, describes what those groups do at the level of single questions, and asks whether the way a student works during the year is related to the way the same student works in the exam. Because a grouping alone says little about how any one student worked, the quantitative part is paired with a reading of individual submission sequences and their code.

Chapter 3

Background

This chapter defines the methods used later, so that the results can be read without looking them up elsewhere. It covers how students are grouped, how the number of groups is chosen, and how the relationship between two groupings is measured. The platform, the course and the data are described in Chapter 4.

The method is deliberately split in two. What is general, and would be the same in any study using these techniques, is here. What is particular to this data, the features, their thresholds and the choice of k, is defined in Chapter 5.

3.1 Hierarchical clustering

Clustering means dividing a set of items into groups so that items in the same group are close to each other and items in different groups are further apart [13]. It is unsupervised: no correct grouping is given in advance, and the method finds structure in the measurements alone.

Each student is described by the values of their behavior features. Those values are measurements: counts of attempts, seconds between submissions, numbers of lines, shares between 0 and 1. They are numbers already, which is what the grouping method needs, since it works from group averages and straight-line distances. Features are measured on different scales, so each one is standardized before grouping: we subtract its mean and divide by its standard deviation, which puts every feature in units of standard deviations from the average student. Without this step a feature measured in seconds would weigh more than a feature measured as a share between 0 and 1. Distance between two students is then the ordinary straight-line (Euclidean) distance between their two sets of values.

We use **agglomerative hierarchical clustering**, which starts with every student alone in their own group and repeatedly merges the two closest groups until one group remains. The result is a tree, called a *dendrogram*, whose branches record

the order of the merges. Cutting that tree at a chosen height gives a grouping. Two properties make this attractive here. The tree is produced once and can be cut at several heights, so the choice of how many groups to keep is made after seeing the structure rather than before. And the tree can be shown, so a reader can see which students merged early and which joined late.

The rule for deciding which two groups to merge is called the *linkage*. Four rules are in common use. **Single** linkage takes the distance between the two closest members of the two groups, **complete** linkage the distance between the two farthest, **average** linkage the average over all cross-group pairs, and **Ward's method** the rise in within-group spread that the merge would cause [25].[1]

Ward's method measures the spread of a group C by its *within-group sum of squares*, the total squared distance of its members to the group mean:

$$\mathrm{WSS}(C) = \sum_{x \in C} \|x - \bar{x}_C\|^2, \qquad \bar{x}_C = \frac{1}{|C|} \sum_{x \in C} x. \tag{3.1}$$

At each step it merges the two groups A and B for which this total rises the least, that is, the pair with the smallest merge cost

$$\Delta(A, B) = \mathrm{WSS}(A \cup B) - \mathrm{WSS}(A) - \mathrm{WSS}(B) = \frac{|A|\,|B|}{|A| + |B|} \|\bar{x}_A - \bar{x}_B\|^2. \tag{3.2}$$

Merging the pair that adds the least within-group spread tends to produce groups of comparable size that are compact around their center, rather than the long chains that single linkage is known to produce. Chapter 5 compares the four rules on the data of this thesis and reports what each one does with it.

3.2 Choosing the number of groups

Hierarchical clustering produces a tree, not a set of groups, so we have to decide where to cut it, that is, how many groups to keep. No test gives the correct answer. What exists are measures of how well a given grouping fits the data, and the usual practice is to compute them for several possible cuts and compare. We use four such measures, described next, rather than relying on one, because each looks at something different and they do not always agree.

Silhouette. For one student i, let $a(i)$ be the average distance from i to the other students of the same group, and let $b(i)$ be the average distance from i to the students of the nearest other group. The silhouette of i is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

[1] General algorithm and its properties: https://en.wikipedia.org/wiki/Ward%27s_method

which lies between -1 and 1 [24].[2] A value near 1 means the student sits much closer to their own group than to any other; near 0 means the two distances are similar; a negative value means the student is closer to another group. The overall average silhouette width is the mean of $s(i)$ over all students. Rousseeuw [24], who introduced the measure, proposes using it to choose the number of groups, and describes the argument as heuristic: one way, in his words, is to take the value for which the average is as large as possible. He also warns against reading a single number on its own, since one distant student can raise the average, and states that when the best value is small it may be better to look for an alternative to clustering. We report the measure and follow that caution rather than treating it as a test.

Calinski-Harabasz index. This compares the spread between groups with the spread inside them, in the form of a ratio, adjusted for the number of groups and the number of students [4].[3] Higher is better.

Davies-Bouldin index. For each group this takes the worst pairing with another group, comparing the spread of the two against how far apart their centers are, and averages the result over all groups [7].[4] Lower is better.

Stability under subsampling. The three measures above describe one grouping of one dataset. They say nothing about whether the same grouping would appear again with slightly different data. To check this we draw a random subset of the students, repeat the whole procedure on it, and compare the grouping obtained with the grouping of the same students in the full data. Agreement is measured with the **adjusted Rand index** [10]. The Rand index counts the pairs of students that two groupings agree about, either by placing both members together or by separating them, as a share of all pairs. The adjusted form subtracts the agreement that would be expected by chance, so a value of 0 means no more agreement than chance and 1 means the two groupings are identical. A grouping that survives resampling is a more useful description than one that does not.

3.3 Comparing two groupings

The final question of this thesis compares two groupings of the same students, one from the coursework and one from the exam. Three quantities are used.

The **chi-square test** asks whether the two groupings are related at all. The students are counted in a table whose rows are the year groups and whose columns

[2] https://en.wikipedia.org/wiki/Silhouette_(clustering)
[3] https://en.wikipedia.org/wiki/Calinski%E2%80%93Harabasz_index
[4] https://en.wikipedia.org/wiki/Davies%E2%80%93Bouldin_index

are the exam groups, and the counts observed are compared with the counts expected if the two groupings were unrelated. A small p-value means the pattern is unlikely under that assumption. It says nothing about how strong the relationship is.

Cramér's V measures the strength of that relationship [6]. For a table of n students with r rows and c columns,

$$V = \sqrt{\frac{\chi^2}{n\,(\min(r, c) - 1)}},$$

which runs from 0, no association, to 1, perfect association.

The **adjusted Rand index**, defined above, is used again here in its original role: it measures how far the two groupings agree on which students belong together, corrected for chance.

These are reported together on purpose. With a large number of students, a small difference can produce a small p-value while remaining unimportant in size, so the strength of an effect and its statistical significance must be read separately [5].

3.4 Conclusion

This chapter has set the tools. Students are grouped with Ward hierarchical clustering on standardized feature values, which builds a tree rather than a fixed set of groups. Where to cut that tree is decided from four measures at once, because no test returns a correct answer and the four do not always agree. Two groupings of the same students are then compared with a chi-square test, Cramér's V and the adjusted Rand index, read together so that a significant result is not mistaken for a strong one.

Two things follow from this for the rest of the thesis. Each of these measures describes a grouping; none of them proves that the grouping is real, so the results in Chapter 5 are reported with their separation and stability values beside them. And nothing here is specific to our data: the features, the thresholds they use and the number of groups we actually keep are decided in Chapter 5, on the data described in Chapter 4.

Chapter 4

Context and Data

This chapter sets out what we study and the data we study it on. It describes the course and the platform the data comes from, the two datasets we use (the coursework across the year and the January exam), and how the data is cleaned and anonymized.

4.1 The course and the platform

The setting is an introductory Python programming course at UCLouvain[1]. Students solve programming tasks by submitting Python code to **INGInious**[2], an *autograder*: a platform that runs each submission against a set of hidden tests and returns a score, without a human grading it by hand. What matters for this thesis is that INGInious keeps the **whole submission history**, rather than only the last attempt. Each submission is recorded with a timestamp, the score the grader gave it, a status, and the exact code file. That history is what lets us look at *how* a student worked, rather than only at their final result.

During the term, the course runs on a weekly rhythm. Students follow the theory, attend exercise sessions led by a tutor, and each week hand in a coding assignment that builds on the material of the previous weeks. They receive feedback from two sources: the tutor, who meets them each week and looks over their submissions to give a short overall comment, and INGInious, which only returns an automatic score on every submission as described above. This weekly coursework is the first of the two datasets we study.

[1] Course pages (same course, different codes by faculty): `LINFO1101` for Computer Science (https://uclouvain.be/en-cours-2025-linfo1101) and `LEPL1401` for Engineering (https://uclouvain.be/en-cours-2025-lepl1401).

[2] INGInious documentation: https://docs.inginious.org/en/latest/what_is_inginious.html.

Across the term the course covers the content of a standard introductory programming course, in six broad parts, from basic program execution to object orientation and linked data structures (Table 4.1).

Table 4.1: The main content of the course, in six parts.

Part	Topics covered
Basics and program execution	Programs, source code and execution; identifiers, variables, values, types, assignment; expressions and statements.
Control flow and functions	Conditionals and loops; functions (parameters, calls, results, scope); specifications and tests; modules.
Data structures	Lists and strings and their operations; tuples, matrices, dictionaries; nested and reference structures; dichotomic search.
Files and exceptions	File handling and input/output; exception handling.
Object-oriented programming	Classes, objects, constructors, methods; class, instance and local variables and their scope; composition, inheritance, encapsulation; polymorphism and dynamic binding; object equality.
Linked data structures	Object references and self-reference; linked structures such as linked lists.

The same platform hosts the course's final exam. The exam runs under a locked-down browser (Safe Exam Browser), so during the exam a student has **no internet and no Python on their own machine**, and a submission only shows **whether the code runs**, not the score or whether it passes the hidden tests. The platform still grades every submission and records the score it gave: what the exam changes is what the student is shown, not what is stored. This is why we can follow how a student's score moved from one attempt to the next even though the student could not see it at the time. We rely on this fact when we read the exam submission sequences in Chapter 6: re-submitting is the only way to test a change, so a long run of submissions is ordinary there.

4.2 Two datasets: the year and the exam

An exam is a single, summative session. To study how students work *as they learn*, we also need the coursework they do across the year. We therefore use two datasets from the same course and platform (Table 4.2):

- the **coursework** of the first quadrimester (2025-2026 Q1): 11 weekly missions covering 99 questions, done at home over the term;
- the **January 2026 exam**: 6 questions solved in one supervised session.

Using both lets us contrast the same student's behavior in two settings, low-pressure practice across the year and a single timed exam, rather than reading everything from one session.

What a mission is, and what a question is. A **mission** is one week of the course, and it has three parts. It opens with a **QCM**, a multiple-choice quiz on INGInious where students select answers rather than write code, scored on the choices they make. It closes, usually on the Friday, with the ***phase de réalisation***: a problem opened for the week that follows, handed in by uploading files to INGInious and reviewed by the academic tutors, with consistent work across these earning extra points on the exam. Between the two sit the **practice questions**. A **question** is one of those: a single programming task with its own statement, its own tests and its own submissions.

This thesis uses the practice questions only. The QCM involves no code, so there is nothing there to follow. The *phase de réalisation* does record attempts, since a student may submit as often as they like up to the deadline, with only the last submission before it counting and going to the tutor; that part of the course is not in the data we hold. What is left is the practice work, and it suits the question asked here: no deadline, and every try recorded. The 11 missions hold 99 practice questions, and those 99 are what Table 4.2 counts. Each is scored by the platform, but that score does not count toward the course grade, so the coursework studied here is practice work, and that is what makes the contrast with the exam meaningful.

Measured per question, not per mission. Each behavior feature is computed from a student's submissions to one question, and that student's value is the median over all the questions they attempted. Three features are counts over the student instead of over one question: how many questions they attempted, and, for the coursework, how many distinct weeks and how many distinct days they were active.

Why one term and one exam session. January is the first sitting of this exam and it follows the coursework term directly, so the two together cover the largest possible set of students who did both. A later session cannot: a student sits the exam again in June or in August only if they did not pass in January, so each further session holds a smaller and more selected group. Those students would also be starting from somewhere else, having already worked through the coursework and already sat the exam once, so a second sitting does not measure the same thing. Chapter 7 returns to what this leaves out.

Table 4.2: The two datasets used in this thesis.

Dataset	Period	Missions / questions	Submissions	Students
Coursework	2025-2026 Q1 (year)	11 missions, 99 questions	235,636	703
Exam	January 2026	6 questions	48,020	581

The exam counts are after the outlier removal of Section 4.3. "Students" counts distinct accounts with at least one submission in that dataset.

In this session the topic of each of the six questions falls in parts 3 to 6 of that content (Table 4.1), from list and matrix manipulation to file handling and object orientation. That is what was asked in January 2026, not a rule: the teachers set the questions for each session, so another exam could draw on any part of the course. Each question and what its function must do is given in Table 4.3.

Table 4.3: The six exam questions (January 2026): topic and what the required function must do.

Q	Topic	What the function must do
q1	Matrix transposition	`transpose(image)` returns a new square binary matrix transposed across the main diagonal, without modifying the input.
q2	Sequence compression	`conway(lst)` compresses runs of equal values into `[count, value]` pairs.
q3	Table allocation	`affecter_tables(tables, reservations)` assigns each reservation to the smallest available table that fits it (first-fit among admissible tables).
q4	File parsing	`longest_word(filename)` returns a 3-tuple `(line, index, word)` for the longest word in a text file, and raises `FileNotFoundError` if the file is missing.
q5	OO inheritance	a `Commercial` class inheriting from `Employe`, with a constructor, a `bonus()` method, and a redefined `salaire()` that combines base salary and commission.
q6	Linked lists	`moyenne_ponderee(notes_etudiant, credits_cours)` computes a weighted average of grades over linked lists, returning the correct value without mutating the lists.

Within the exam, the number of submissions falls from the first question to the last (Table 4.4): fewer students reach the later questions, which is worth keeping

in mind when reading per-question results.

Table 4.4: Exam submissions per question (January 2026, outlier accounts removed).

Question	q1	q2	q3	q4	q5	q6	Total
Submissions	9,899	11,910	9,172	8,394	5,760	2,885	48,020

4.3 Anonymization, ethics and outlier removal

Where the data comes from and under what permission. The submission records used in this thesis were provided by its supervisor, Prof. Kim Mens, who authorized their use for this work. They reach us **already anonymized**: each student appears only as a hash, with no name, student number or other identifying detail, so no result in this thesis can be traced to a person. We do not perform the anonymization ourselves and we never hold the identifying data.

Showing student code. Chapter 6 and Appendix C reproduce extracts of real submissions, so that the reading of each case can be checked against the code it rests on. Each extract carries the anonymous hash and nothing else. One residual risk should be stated: Ihantola et al. [11] warn that in programming-process data a student can in principle be recognized from timestamps, comments or self-chosen variable names. We show short extracts, tied to no identifier we hold, but we do not claim the risk is zero.

Before any analysis we remove a small number of **outlier accounts** whose exam activity cannot correspond to a single genuine sitting. The exam took place on 22 January 2026 in two slots on the same day (roughly 09:00 to 12:00 and 12:15 to 15:15), each up to one hour longer for students entitled to accommodations, plus a short launch buffer. A single genuine sitting therefore falls inside 09:00 to 16:30 and lasts at most about 4 hours 15 minutes. From the timestamps, we flag and drop any account that

- has a submission on a day other than 22 January 2026;
- has a submission outside the exam hours of that day (before 09:00 or after 16:30); or
- spans more than 4 hours 15 minutes from its first submission to its last (the longest possible sitting, accommodations and a short launch buffer included).

This removes 7 accounts, taking the exam from 588 to the 581 students in Table 4.2.

What we call the exam grade. Throughout this thesis, a student's *exam grade* is the mean of their final score on each of the six exam questions, on a scale of 0 to 100, with a question they never attempted counting as 0. It is the autograder's

score, not the mark the student received for the course: it is not on the usual 0 to 20 scale, and it does not include the bonus points earned by working consistently on the *phase de réalisation*. Two consequences follow. A student who ran out of time before the last questions is scored on all six regardless, so the grade reflects how far they got as well as how well they did. And the per-question figures in Section 5.7 are computed over the students who attempted that question, which is a different denominator.

Where these accounts come from. They are staff and test accounts, as confirmed by the course supervisor. Their activity sits before or after the exam day, in the pattern of setting up or checking the questions: one spans 39 days, another made 175 submissions. All 7 also appear in the coursework data, and they are excluded there too. The process also looks beyond timing: it checks that each submission's recorded score, submission number and hash match its stored file, that the status and the score agree, and that every timestamp is readable. These checks pass on all 48,020 kept submissions, so the timing rules above are the only ones that remove accounts. For the coursework, which is spread over the term by design, timestamps are used only to reconstruct each submission sequence, and a gap longer than a day between two submissions of the same question is treated as returning across sessions rather than as working rhythm (Appendix A).

The two student sets overlap but are not identical: not every exam-taker did the coursework on this platform, and not every coursework student sat this exam. Whenever we compare a student's year behavior with their exam behavior, we use only the **569 students present in both datasets** (the *linked set*). All the grouped results in Chapters 5 and 6 are computed on these 569 students.

4.4 Pipeline and reproducibility

The analysis runs as a chain of small, separate steps: extract the submissions, clean and audit them, build the per-student features, clean the feature set, group the students, then describe and compare the groups. Each step is a separate script that writes its own intermediate file, so the data can be inspected at every stage rather than hidden inside one large program. The exact thresholds and formulas are collected in Appendix A. The public repository[3] records which script produces which output, so any figure or number in this thesis can be traced back to the code that produced it.

[3] https://github.com/Lord-Melflam/Master_Thesis_Francois_Meli_2026

4.5 Conclusion

This chapter has described what we study and on what data. The setting is one introductory Python course at UCLouvain, taught on INGInious, an autograder that keeps every submission a student makes. Two datasets come from it: the coursework across the term, and the January exam that comes after it, taken under a locked-down browser where a submission tells the student only whether the code runs. We removed 7 staff and test accounts, and the analysis uses the 569 students who appear in both datasets.

Two important points for the chapters that follow. The exam setting tells us how its submissions should be read: with no score shown and no way to run code locally, re-submitting is the only way to check a change, so a long run of attempts is ordinary there and does not mean the same thing as during the term. And every grouped result in Chapters 5 and 6 rests on the same 569 students, so the two settings are always compared on the same people.

Chapter 5

Quantitative Results

Chapter 2 showed that a finished program says little about how it was produced, and Chapter 3 set out the methods used to group students. This chapter applies them. It groups students by how they worked, which answers RQ1, and it compares the two groupings, which is the first half of the answer to RQ3. The second half is in Section 6.5, where the same students are followed one by one across the two settings, the coursework year and the January exam. Chapter 6 goes down to single questions and answers RQ2.

The **final grade** takes no part in forming the groups. It is not one of the features, and it appears only afterwards, beside each group, as a point of comparison. The **score of a single submission** is a different matter, and three of the features below do use it: whether a try raised the score, whether a large edit failed to raise it, and whether a small edit raised it sharply. **So the grouping is not score-free; it is final-grade-free.**

All numbers in this chapter are computed on the **569 students** who are present in both the year (coursework missions) and the January exam.

We study behavior at two levels, and it helps to name them clearly:

- An **episode** is one student working on *one* question: the whole ordered sequence of their submissions to it, including how the code and the score change from one try to the next. An episode is a single, concrete stretch of behavior.

- A **profile** is a student's *usual* way of working, taken across all of their questions (in practice, the median of each feature over their questions). A profile is the tendency behind many episodes, not a single behavior.

Both names are taken from the literature on programming-process data, and both are used here with the operational meaning just given. Liu and Paquette [16] use *episode* for a run of submissions in autograder logs: they define a debugging episode

as a sequence in which a student receives the same test error in several submissions in a row. Ours is bounded by the question instead of by an error message, because at our exam a submission only reports whether the code runs, so we do not have the error labels their unit needs. *Profile* is the usual word for a student's typical way of working: Fraser [8] builds profiles at two levels of detail, one per work session and one per student over a semester, which is the same split we make between an episode and a profile. Baker and Inventado [3] describe the same choice as a choice of level, between clustering students and clustering student actions.

One point of vocabulary, to avoid a common confusion: here a *profile* belongs to one student, and the groups we report later (E1 to E4 for the exam, Y1 to Y3 for the year) are sets of students whose profiles are similar. This chapter groups students by profile. Single episodes, and the case studies built from them, are in Chapter 6.

5.1 The features we grouped students on

For each student we use behavior features only: how many questions they attempt, how many attempts they make per question, how fast they resubmit, how long they pause, the typical gap between tries, how often a new try improves the score, how much code changes between two submissions, how often a *large* edit does not raise the score, how often a *small* edit is followed by a large score gain, how many lines of code they write, and how many distinct programming concepts they use. The last two, the size of the final program and the number of concepts it uses, are code-quality measures: they describe the code a student ends up with, rather than only how they worked to get there. We read them on the last submission. Reading them on an earlier one would mix two things: what the student wrote, and how far through the task they were at that moment. How far they had got is worth knowing, but it is a different measurement from the one we want here. Each feature is computed **per (student, question)** first, then combined into one value per student by the **median** (robust to outliers). A student does not behave identically on every question, so the median reflects their *typical* behavior and is not thrown off by one unusual question; this is why we speak of a *profile* (a tendency across questions), not a fixed label. Two of these can look like opposites and are not. *Tries that improved* counts every submission that raised the score, whatever its size. *Edits, no score gain* looks only at the largest edits, those in the top tenth by size, and asks whether that effort paid off. A student can score high on both. Table 5.1 gives the exact meaning and computation of each feature.

Why these features. They are meant to describe how a student went about the work, not how well it turned out. They fall into five groups: how much a student attempted (questions attempted, attempts per question), the pace of the

work (quick resubmissions, long pauses, the typical gap between tries), whether the tries paid off (tries that improved, edits with no score gain, small fix with a big gain), the code that came out of it (code lines, comment share, distinct concepts), and, for the coursework only, how the work was spread over the term (active weeks, active days).

Several quantities were computed and then deliberately kept out of the grouping, because each one is the score itself or is computed from it: the best and the final score on a question, the score gained between the first try and the best one, the share of questions passed or fully solved, and the exam grade itself. They are used only afterwards, to describe the groups once they exist. The time a student took to reach a passing score was left out as well: it depends on reaching the pass mark, and it has no value for a student who never did.

Moreover, the code-quality features are there because a description built only from timing and counts would say nothing about what the student actually wrote. The share of comment lines is kept at this stage; Section 5.2 shows that it is almost the same for every student in the coursework and drops it there, while keeping it for the exam. Whether two features carry the same information is not decided by hand, and Section 5.2 removes the redundant pairs.

Table 5.1: The behavior features, with how each is computed.

Feature	How it is computed (per question, then median over the student's questions)
questions attempted	number of distinct questions with at least one submission
attempts per question	number of submissions to the question
quick resubmissions	share of gaps that are "quick" (gap $\leq$ 10th percentile of gaps)
long pauses	share of gaps that are "long" (gap $\geq$ 90th percentile of gaps)
gap between tries	mean seconds between consecutive submissions
tries that improved	share of consecutive submissions where the score went up
lines changed / edit	median edit size (changed lines between consecutive submissions)
edits, no score gain	share of *large* edits (in the top tenth of edit sizes over the whole dataset) that did *not* raise the score
small fix, big gain	share of *small* edits ($\leq$ the median edit size over the whole dataset) that raised the score by at least 50 points out of 100
code lines	non-comment lines of the final submission
comment share	comment lines / non-blank lines of the final submission
distinct concepts	number of the 8 AST concept types (loop, conditional, function, class, comprehension, exception, with, lambda) present in the final submission. Counting them needs the code to parse, so a submission with a syntax error is left out of the median; a student none of whose submissions parse is counted as 0
active weeks (year only)	number of distinct calendar weeks with a submission
active days (year only)	number of distinct calendar days with a submission

In this table, a *gap* is the number of seconds between two consecutive submissions of the same question, an *edit* is the number of lines changed between two consecutive submissions (via `difflib`), and *final* is the student's last submission on a question. The two features marked *year only* exist for the coursework alone, because the exam is a single session.

Thresholds are computed from the data, not fixed by hand. A few features need a **cut-off**, a value that separates two cases, such as the gap length above which we call a pause long. We derive each cut-off from this dataset instead of choosing a round number. A fixed number would not mean the same thing in both settings: a ten-second gap between two submissions means something in a timed exam, and very little in the coursework, where the work is spread over weeks.

Each cut-off is therefore taken from the distribution of the setting it applies to.

- **Pass** = score ≥ 50. This one is not tuned: it is the course's own pass mark.
- **Quick / long** gap: a gap between two submissions is "quick" if it is in the fastest tenth of all gaps, and "long" if it is in the slowest tenth. The two cut-offs are the 10th and 90th percentiles of *every* gap in that setting, pooled over all students and all questions, and the exam and the year are computed separately: quick $\leq$ 12s and long $\geq$ 243s in the exam, $\leq$ 11s and $\geq$ 273s in the coursework. If two submissions are more than a day apart, the student has left and come back. That gap says nothing about how fast they work, so it is not counted. A question with a single submission has no gap at all and contributes nothing to these two features. The two do not add up: a gap in the middle is neither quick nor long, and most gaps are in the middle. The 10th and 90th percentiles are not tuned. We fixed them before looking at any result, as a plain way of saying *unusually fast* and *unusually slow*. Chapter 2 notes that Jadud searched for the parameters of his own measure, keeping the values that made students' scores as different from each other as possible; we did not do that here.
- **Small / large** edit: an edit is "small" if it is at or below the median edit size and "large" if it is in the top tenth of edit sizes. Both cut-offs are taken from every edit in that setting, again pooled over all students and all questions. The two are not symmetric on purpose, because they answer different questions: "small" has to mean *no bigger than usual* for that setting, while "large" has to mean *unusually big*. In the exam this gives small ≤ 1 line and large ≥ 6; in the coursework, ≤ 2 and ≥ 9.
- **Big score gain** ≥ 50: a jump of at least half of the 100 marks. We use an absolute jump rather than a percentage rise because many sequences start at a score of 0, where a percentage rise is undefined, and near 0 it explodes: a move from 1 to 6 would count as a 500% rise. The score just before such a jump is 0 in 46% of the exam cases and 64% of the year cases, so a relative rule would have nothing to work with on about half of them. A rise of at least 50 points is also what marks a *breakthrough* episode in Chapter 6, and Section 6.6 reports how common that shape becomes if the rise is set to 25 or to 75 points instead.

The exact formula and the derived value of each threshold are given in Appendix A; the point is that no cut-off is an arbitrary hand-picked number.

5.2 Cleaning the features before grouping

Some features carry almost the same information (for example the number of active days and the number of active weeks). If we keep both, the grouping counts that information twice. So before grouping we clean the feature set in two steps, applied the same way to the year and to the exam:

- **Drop near-constant features.** A feature that is almost the same value for nearly every student cannot separate students. The share of comment lines is such a feature in the coursework: the median is zero for 567 of the 569 students, so it cannot separate one student from another and it is not used to form the year groups. In the exam it is kept, because there the median is zero for 376 students and above zero for the other 193, which is variation the rule has no reason to discard. The rule is the same in both settings; the data is not.
- **Drop redundant features.** When two features are very strongly correlated ($|\rho| \geq 0.80$, Figures 5.1 and 5.2), we keep only one of them, so the same information is not counted twice. Which one goes is not decided by hand: we repeatedly drop the feature that is most correlated with all the others still in the set, until no pair is left above the cut. In both figures a cell is dark when two features carry almost the same information, pale when they do not, and the pairs we drop are the dark ones off the diagonal.

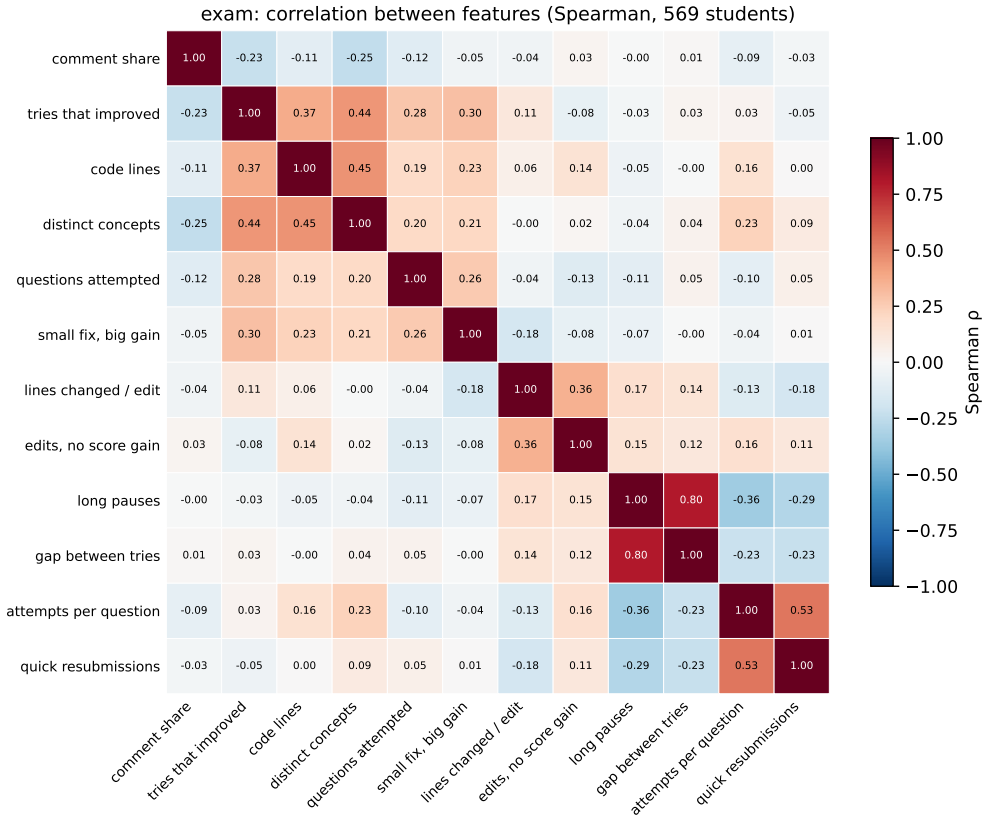

Figure 5.1: Correlation between features (exam).

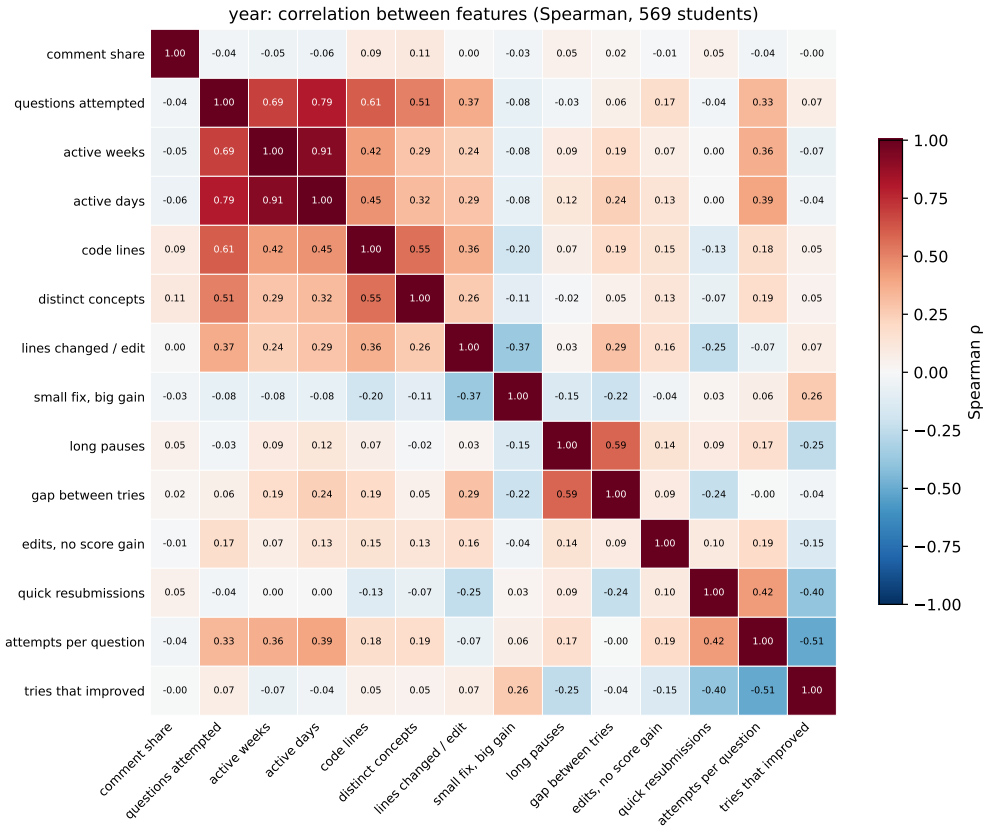

Figure 5.2: Correlation between features (year).

What we remove is **not the same** in the year and in the exam: the exam drops the long-pause share (it duplicates the gap between tries, $\rho = 0.80$), while the year drops the number of active days (it duplicates the number of active weeks, $\rho = 0.91$) and the comment share (near-constant). Both pairs measure the same thing twice: a student who pauses often has longer gaps between tries, and a student who works on more days works in more weeks. The two settings do not have the same redundant structure because they run on different time scales: the exam is one sitting, with gaps measured in minutes and no spread over days or weeks at all. Cleaning the features also makes the exam grouping more stable when we re-run it on random subsamples of the students (Figure 5.3), and it is higher for the year at the chosen $k = 3$ as well, about 0.60 against 0.43. We measure this stability by re-clustering many random 80% subsamples and recording how often the same students end up grouped together, summarized by the adjusted Rand index[1] (ARI), which runs from 0 (no better than chance) to 1 (the same grouping

[1]The Rand index (RI) is the share of student *pairs* two groupings agree on (together in both,

every time).

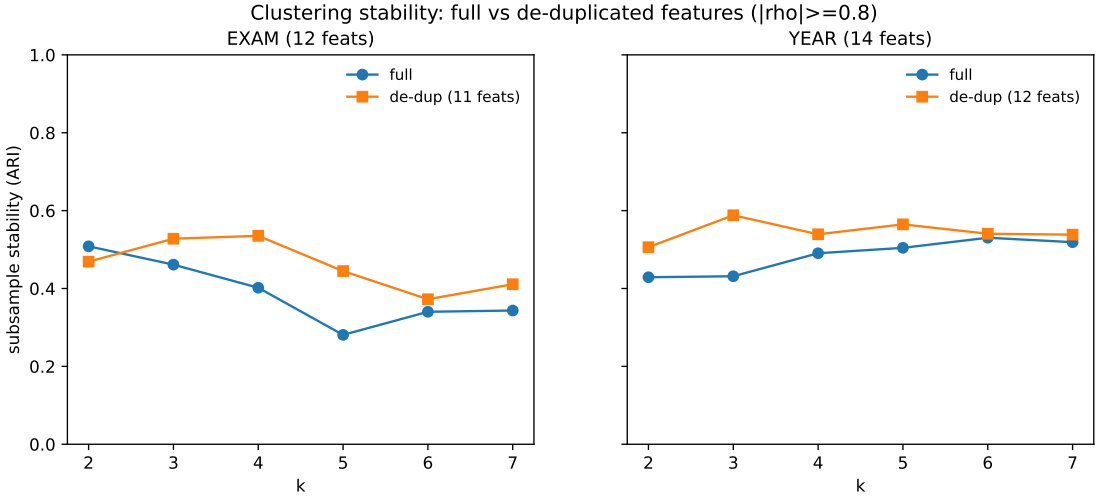

Figure 5.3: Grouping stability before and after cleaning the features, for the exam (left) and the year (right).

5.3 Linkage and the number of groups

We group students with hierarchical clustering on the standardized features, using Ward linkage [25]. The method, the four linkage rules and the measures used below are defined in Chapter 3; this section reports what they do with our data.

We chose Ward after comparing the four linkage rules on our own data (Figure 5.4). We expected some of the rules to give very uneven groups. The feature values change gradually from one student to the next, so there is no empty space separating one set of students from another, and a rule that merges on the closest pair of members will then take the students one at a time. That is what we see: with single, average and complete linkage each merge usually just adds one more student to the same large group (single linkage shows the staircase pattern known as chaining), so a cut leaves one very large group and many groups of one or two students. Only Ward divides the 569 students into groups of comparable size, which is what we need in order to describe behavior rather than to isolate outliers.

The four measures used below to choose k can also be computed under each of the four rules, and they are, in Appendix B. Three of them come out *better* for the other rules than for Ward. The size of the groups explains why: single

or apart in both); the adjusted Rand index corrects it for chance, $\mathrm{ARI} = (\mathrm{RI} - \mathrm{E}[\mathrm{RI}])/(\max \mathrm{RI} - \mathrm{E}[\mathrm{RI}])$, so 0 is chance level and 1 is identical [10]. https://en.wikipedia.org/wiki/Rand_index

linkage puts 566 of the 569 exam students in one group and leaves three on their own, and a grouping like that scores well on separation precisely because almost everyone is inside one large group and the few who are not sit far away. Only Calinski-Harabasz, which weighs the spread between groups against the spread inside them, prefers Ward.

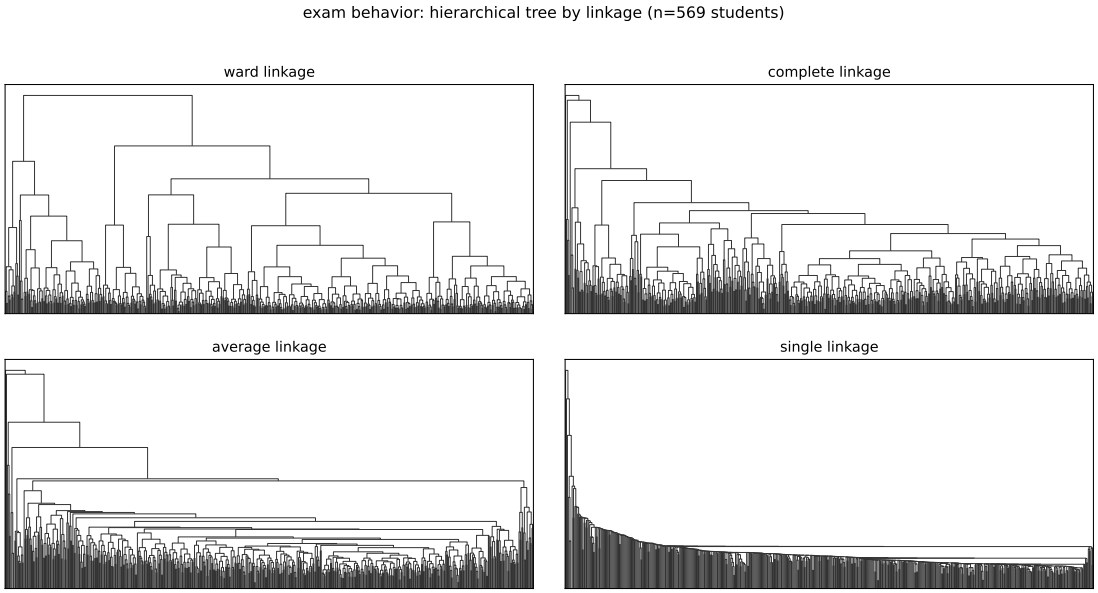

Figure 5.4: The same 569 students clustered with four linkage rules (exam features).

The year data behaves the same way (Figure 5.5): again only Ward gives balanced groups, so we use Ward for both datasets.

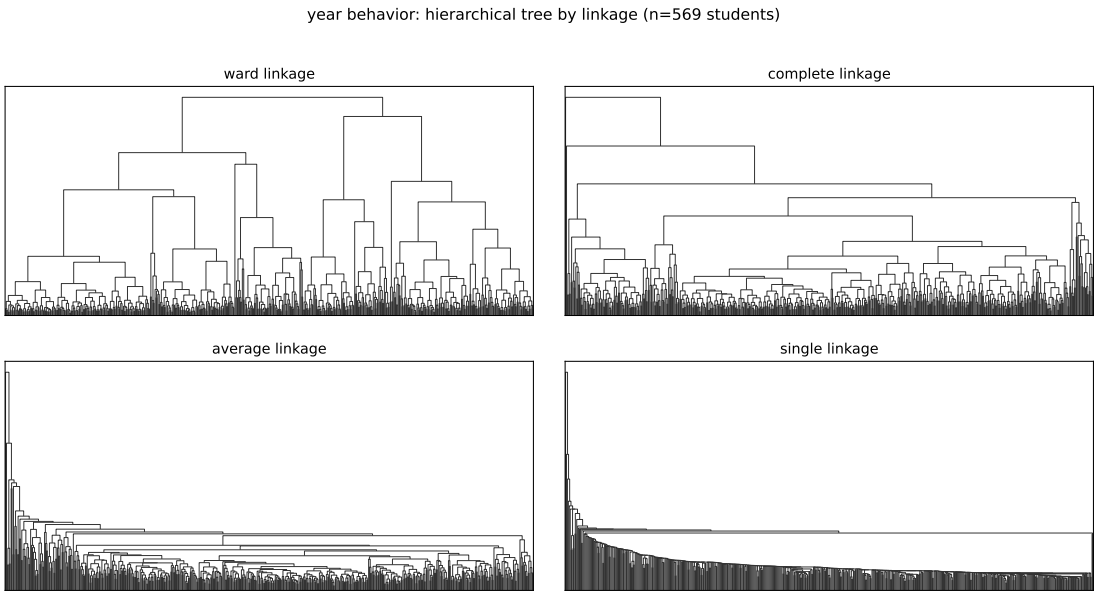

Figure 5.5: The same four-linkage comparison on the year (coursework) features.

The tree can be cut at different levels to give a different number of groups k. We do not pick k by eye and we do not pick it for teaching convenience. We choose k from the data, using four standard measures across $k = 2$ to 8: the silhouette[2], the subsample stability (how often the same students stay grouped together when we re-cluster random 80% subsamples), the Calinski-Harabasz index[3], and the Davies-Bouldin index[4] (Figure 5.6), run separately for the exam and the year. In that figure the Calinski-Harabasz and Davies-Bouldin indices are rescaled onto the same 0 to 1 axis so that all four measures can be read together, and the dotted line marks the k we use. We expected these measures to agree on a small k if the behavior has real group structure.

[2]The silhouette of a student compares how close they are to their own group versus the nearest other group; it runs from -1 to 1, higher meaning tighter and better-separated groups. We report the average over all students [24]. https://en.wikipedia.org/wiki/Silhouette_(clustering)

[3]The ratio of the spread *between* groups to the spread *within* groups; a higher value means groups that are tighter and further apart [4]. https://en.wikipedia.org/wiki/Calinski%E2%80%93Harabasz_index

[4]The average, over all groups, of how similar each group is to the group most like it; a lower value means better-separated groups [7]. https://en.wikipedia.org/wiki/Davies%E2%80%93Bouldin_index

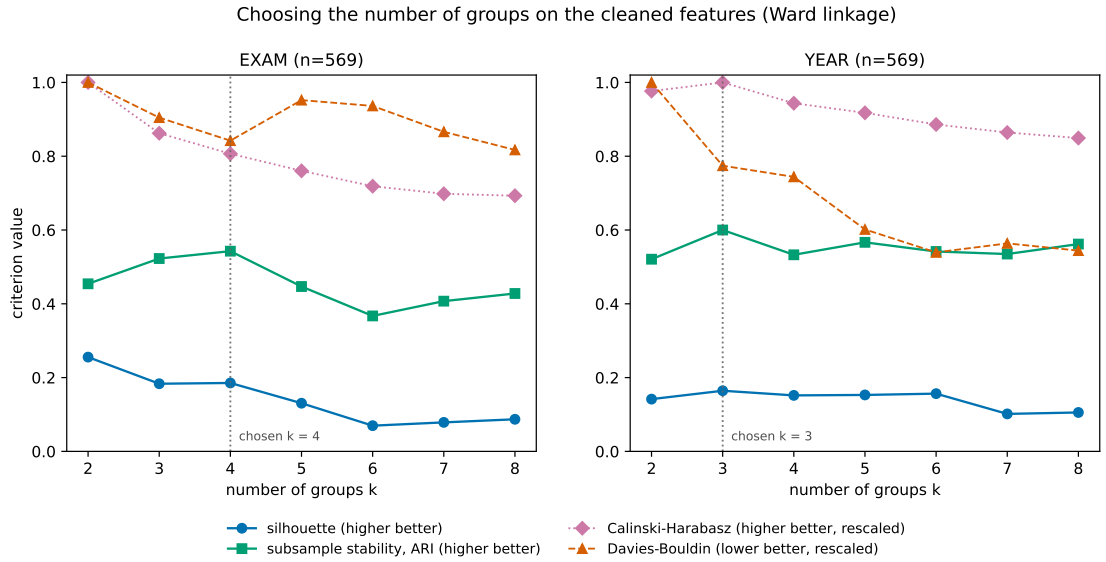

Figure 5.6: Choosing the number of groups on the cleaned features, for the exam (left) and the year (right).

The two settings do not have to be cut into the same number of groups. Each grouping is formed on its own data, and the measures that compare them later, Cramér's V and the adjusted Rand index, are defined for tables whose sides differ. Section 5.6 also repeats the comparison with the year cut into two, three and four groups, and the result does not change.

For the year the measures agree on **three groups**: the silhouette, the stability and the Calinski-Harabasz index are all best at $k = 3$ (Table B.2). For the exam they do not all agree (Table B.1). The silhouette is highest at $k = 2$ (0.256 against 0.185 at $k = 4$), while the stability and the Davies-Bouldin index are best at $k = 4$ (0.54 against 0.45, and 1.82 against 2.16). On the silhouette alone, which is how Rousseeuw [24] suggests reading it, the exam would be cut into two groups. However, we use **four**, because that cut is the one that comes back when we re-cluster random 80% subsamples (so the highest stability), and a grouping that survives resampling is worth more than one that scores higher on a single measure of separation. The full table is in Appendix B.

5.4 The exam groups

If these groups reflect behavior rather than the score alone, we expected each one to show a *distinct feature signature*, not merely a high or low grade. The four exam groups are shown in Figure 5.7. Each group is described by its feature signature:

how far its students sit from the average of all students on each feature, in standard deviations. The grade is shown next to each group for reference only.

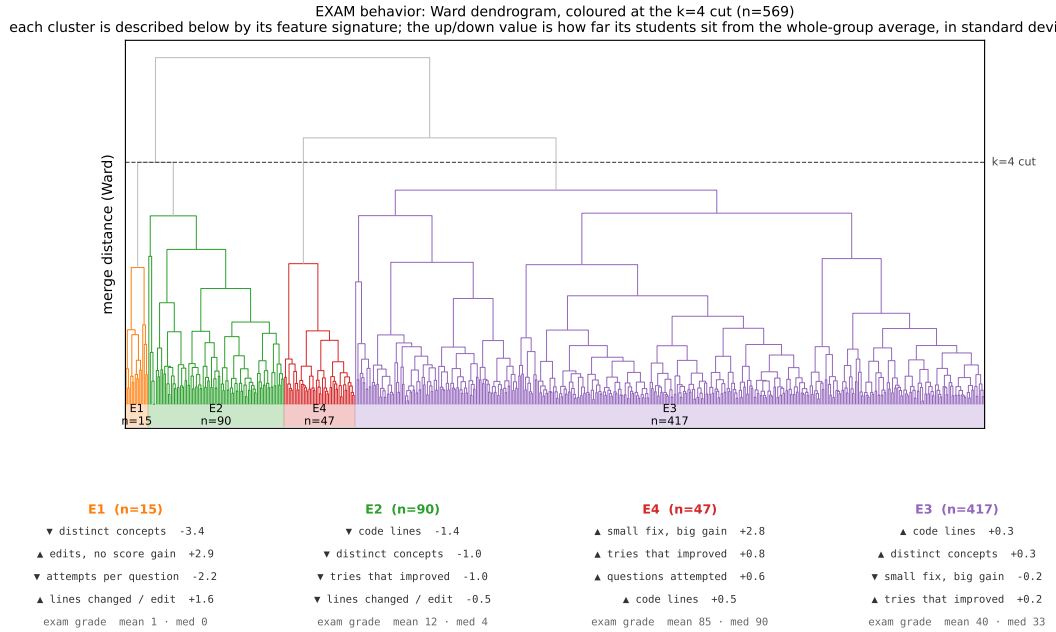

Figure 5.7: Exam groups (Ward tree cut at four groups, $n = 569$). Each colored block is one group, described below by its feature signature.

The four groups read as follows:

- **E1** ($n = 15$): a small but distinct group. These students made *few* attempts per question, but each edit was *large*, and a very high share of them fell under "edits, no score gain". Their code often did not even run: about a third of their final submissions (32%) had a syntax error, against 5% in the large middle group and none in the top group, and 3 of the 15 never produced a single submission that parsed. What did parse used very few programming concepts (median 1, against 3 over all students) and was short (median 13 lines, against 22). They rewrote large parts of their code and the score almost never moved; mean exam grade $\approx 1/100$ (median 0). We keep this group despite its size: it is not a single outlier (the point of profiling behavior is what the *features* say, not the score or the count), and its signature is a clear, extreme pattern of large rewrites that leave the score where it was.
- **E2** ($n = 90$): wrote little that worked, with few code lines, few concepts, and rarely improved from one try to the next. Mean exam grade 12/100 (median 4).

- **E3** ($n = 417$): the large middle group, close to the average of all students on every feature. Mean exam grade 40/100 (median 33).
- **E4** ($n = 47$): often scored very high after a small, targeted edit ("small fix, big gain"), attempted more questions, and improved more often. Mean exam grade 85/100 (median 90).

Chapter 6 takes two students from each of these four groups and follows their submissions one by one.

The same groups are shown as a heatmap in Figure 5.8.
How to read it: each column is a student, ordered by the tree, and the colored band on top marks the four groups; each row is a feature; a cell is red when that student is above the average of all students on that feature and blue when below. So a group whose columns form a solid red or blue band on a row is uniformly high or low on that feature, and that band *is* the group's signature. Reading the rows within each block: the small **E1** block is a deep **red** band on "edits, no score gain" and a deep **blue** band on "distinct concepts" (edits that rarely raise the score, on code that uses very few concepts); **E2** is blue on "code lines" and "distinct concepts" (wrote little that worked); **E4** carries the eye-catching **red** band on "small fix, big gain", the feature that defines the high-scoring group; and the large **E3** block is mostly pale on every row, exactly what an average middle group looks like.

Figure 5.8: Exam groups as a clustermap (features × students). The colors inside each colored block are that group's feature signature.

In our data the exam groups line up with the exam grade: the mean grade rises steadily from E1 to E4 (about 1, 12, 40, 85 out of 100). Because the groups were built without using the grade, this alignment is something we observe, not something we built in.

The groups are still more than the grade under another name. E1 and E2 both score low (about 1 and 12), yet they behave differently: E1 makes few but large edits that almost never raise the score, and about a third of its final submissions do not even run, while E2 writes little code and seldom improves from one try to the next. The grade alone cannot tell these two apart, but their behavior can. And, as the next section shows, the year groups barely differ in grade at all, so grouping students on behavior does not simply recover the score.

5.5 The year groups

The year splits into three groups (Figure 5.9):

- **Y1** ($n = 155$): fewer questions attempted, fewer active weeks, fewer attempts, and less code. Mean exam grade 33 (median 20).
- **Y2** ($n = 93$): many quick resubmissions and more attempts, but fewer of them improve the score. Mean exam grade 35 (median 28).
- **Y3** ($n = 321$): more questions attempted, more active weeks, and more code. Mean exam grade 42 (median 35).

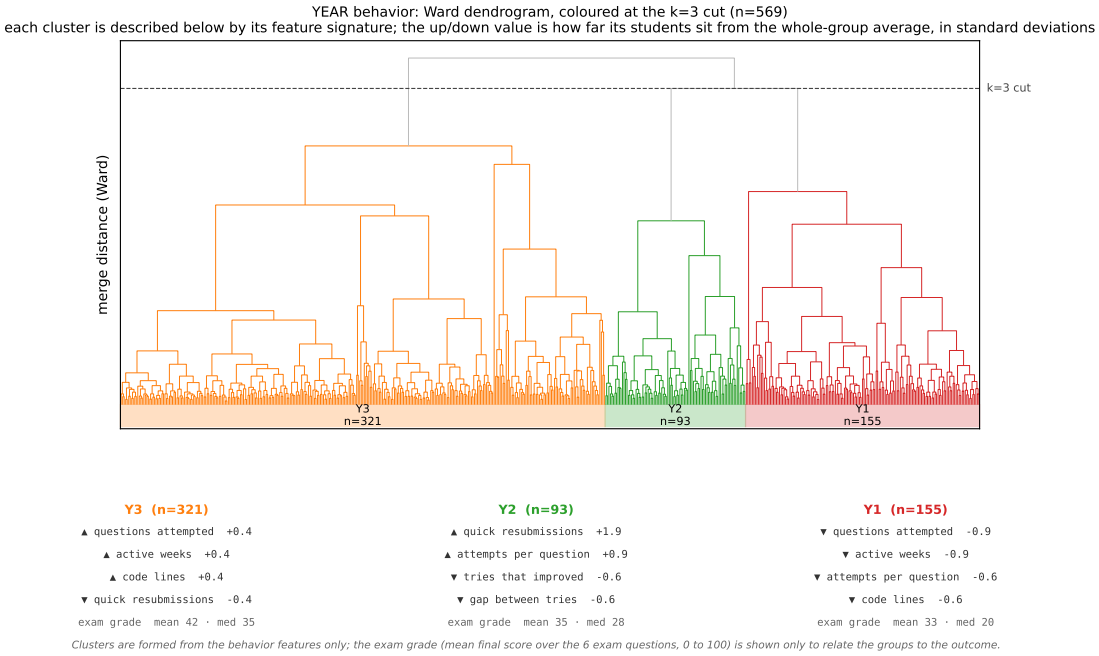

Figure 5.9: Year groups (Ward tree cut at three groups, $n = 569$), described by their feature signatures. The exam grade is shown for reference only.

The year clustermap (Figure 5.10) is read the same way as the exam one. Two things stand out. First, the color bands are **more spotty and uneven** than in the exam: there are fewer solid, full-width bands within a group, which is the visual sign that the year groups are less sharply separated. Second, the concentrations that do appear match the signatures: the Y3 block (most questions attempted) is a warm band on "questions attempted", "active weeks" and "code lines" and cool on "quick resubmissions", while the Y1 block (fewest questions attempted) does the

opposite: it is blue on those same three rows. The Y2 block carries a strong red band on "quick resubmissions" and a lighter one on "attempts per question". One row, "edits, no score gain", is pale almost everywhere: this feature barely varies during the year, so it does little to separate students.

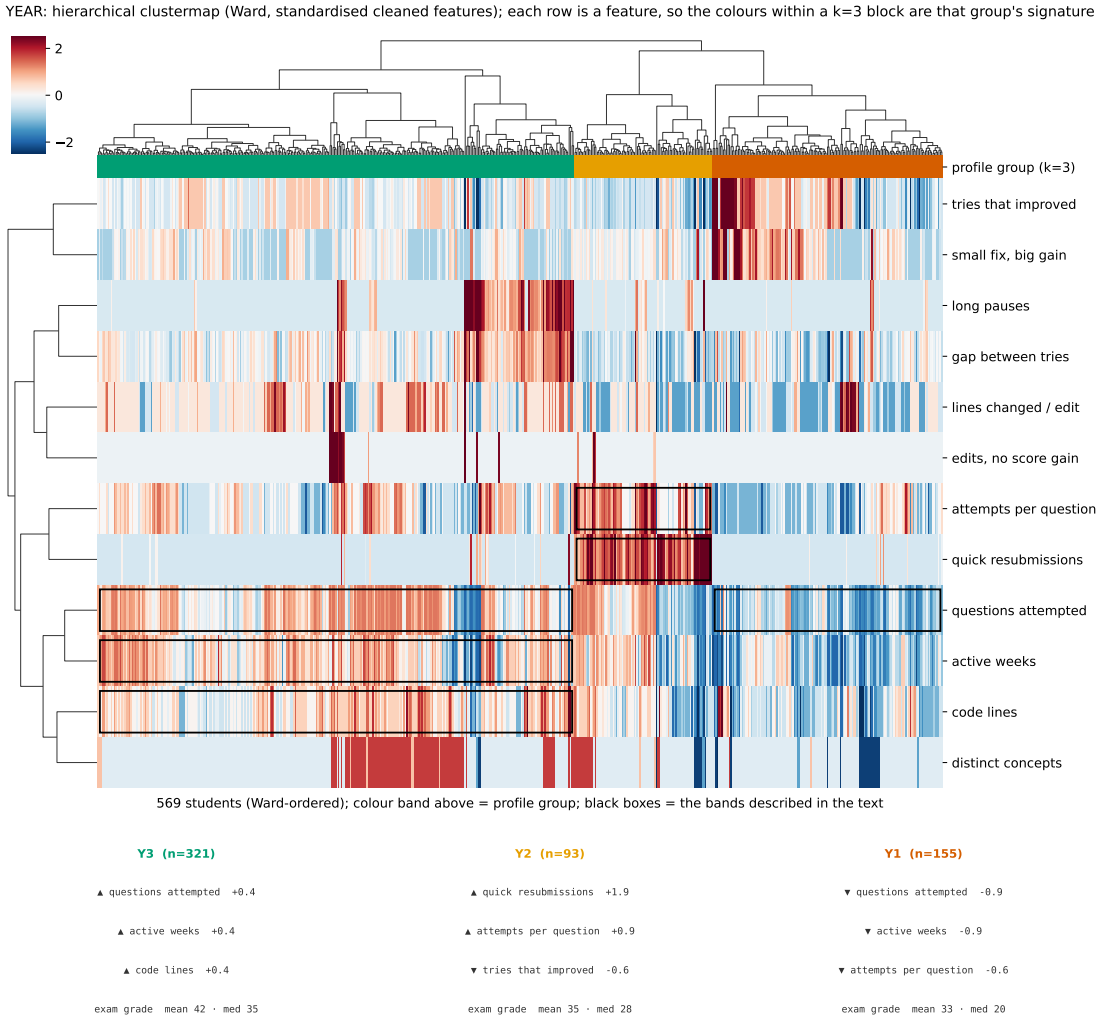

Figure 5.10: Year groups as a clustermap (features × students). Compared with the exam (Figure 5.8), the bands are more spotty and uneven, the visual counterpart of the weaker, more grade-flat year structure.

The year groups differ far less in grade than the exam groups do: their mean exam grades run only from 33 to 42, against 1 to 85 for the exam. So the way students work during the year is only **weakly** related to the exam grade. There is a mild gradient, since the year group that attempted the most questions scores a

little higher, but nothing like the strong split the exam shows. Section 5.6 comes back to why.

5.6 Year groups against exam groups

Each of the 569 students is in one year group and in one exam group, so we can ask whether the two groupings match. If the way a student works during the year carried over to the exam, we would expect each year group to go mostly into one exam group. That is not what we see: they barely match. Table 5.2 shows, for each year group, where its students land in the exam.

Table 5.2: Where each year group lands in the exam, $P(\text{exam} \mid \text{year})$; rows sum to 100%. The bottom row is the base rate $P(\text{exam})$ for comparison.

	E1 (g1)	E2 (g12)	E3 (g40)	E4 (g85)
Y1	6%	23%	61%	10%
Y2	3%	22%	69%	6%
Y3	1%	11%	80%	8%
base rate	3%	16%	73%	8%

Students from year group Y1, the group that attempted the fewest questions, are a little more likely to land in the lower exam groups (E1 and E2 together: 29%, against a base rate of 18%, that is 105 of the 569 students), and students from Y3, which attempted the most, are less likely to (12%). But the difference is small, and every year group still sends most of its students to the big middle exam group E3. We measured the strength of the link on the full table (Figure 5.11). A chi-square test says the two groupings are not independent ($\chi^2(6) = 28.6$, $p < 0.001$), but that only means the pattern is unlikely to come from chance; it does not say the link is strong. The link itself is small: Cramér's $V = 0.159$,[5] which squared is about 2.5% of what a table of this shape could show. The adjusted Rand index is 0.07, so the two groupings agree barely more than chance would. With 569 students almost any difference comes out significant, so what counts here is how big it is [18].

We therefore read the year-to-exam link as **weak**: knowing a student's year group tells us little about which exam group they fall into.

[5] Cramér's V says how strongly two groupings go together, from 0 to 1: $V = \sqrt{\chi^2/(n\,(\min(r,c)-1))}$ [6]. Cohen [5] puts the usual thresholds at 0.10 small, 0.30 medium and 0.50 large for a 2×2 table, divided by $\sqrt{r-1}$ for a larger one, where r is the smaller side of the table. Ours is 3×4, three year groups by four exam groups, so the thresholds become 0.07, 0.21 and 0.35.

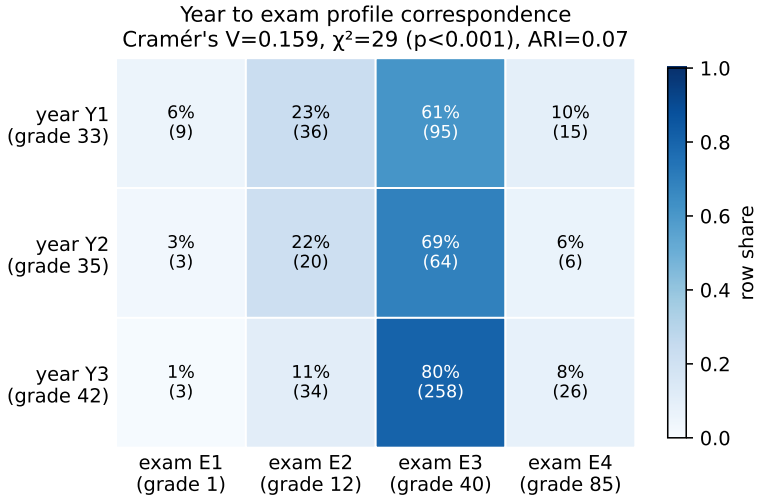

Figure 5.11: Correspondence between the year groups and the exam groups on the same 569 students. The association is weak ($V = 0.159$) even though it is statistically significant.

We checked that this result does not depend on how many year groups we use. Whether we split the year into two, three, or four groups, the groups still barely match the exam groups (Cramér's V between 0.13 and 0.21, adjusted Rand index near zero). The full $P(\text{exam} \mid \text{year})$ and $P(\text{year} \mid \text{exam})$ tables for each year k are in Appendix B (Tables B.3 to B.5); they show that a finer year split only cuts the big year group into thinner slices that still land in the same exam profile. So the weak link does not come from our choice of k. A single-student view of the same finding is given in Chapter 6.

Why might the two settings differ so much? We cannot tell from these data, but a few plausible reasons fit the pattern. The coursework is done at home, with no time limit, and a student can get help or reuse a solution from a classmate or from a previous year; the exam removes all of this, since it runs under a locked-down browser in a single supervised session. So the year work may look easier and more uniform than the exam partly because the year allows shortcuts the exam does not, and not only because students learned. These data cannot show whether a student reused code or got help.

5.7 Performance across the exam questions

Separately from the grouping, we looked at the six exam questions themselves, using the scores, on the same 569 students. Three common beliefs can be checked directly.

Does performance fall along the question order? One might expect the later questions to be the harder ones. We measured each question's *performance* by the mean and median final score and the pass rate over the students who attempted it, and we do not read these numbers as a measure of intrinsic difficulty (a low rate could come from the topic, the order, or the time left; we cannot tell from these data). Performance does not fall along the question order (Figure 5.12): question 5 has the *highest* performance (pass rate 68%, mean score 64), while questions 4 and 6 have the *lowest* (mean scores 35 and 33, pass rates 36% and 32%). So the pattern does not follow the question number. (Fewer students even reached question 6: 300, against about 550 for question 1.)

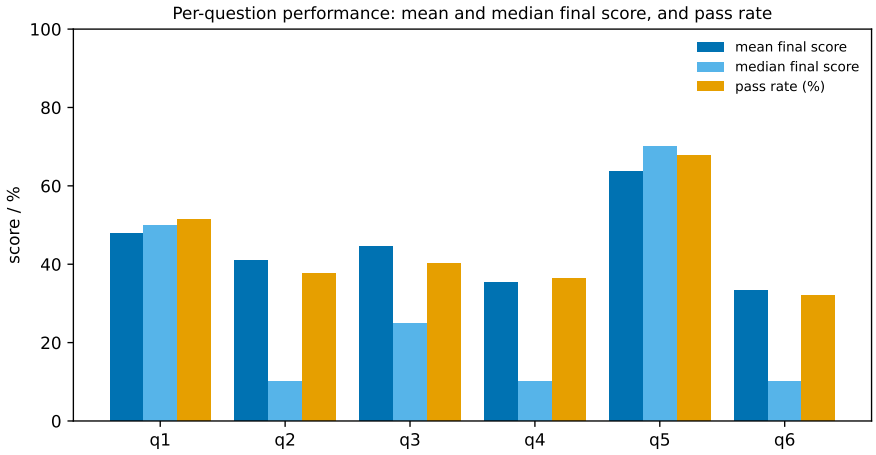

Figure 5.12: Per-question performance: mean and median final score, and pass rate. Performance does not fall along the question order.

Does failing one question mean failing another? A common student belief might be "if I cannot solve question 1, I will fail question 6." We tested it with the correlation between each pair of questions' final scores (Figure 5.13). If the belief described a special chain, or if only neighboring questions were linked, we would expect an uneven pattern. Instead, every pair is positively and *similarly* correlated (r from 0.35 to 0.65). Neighboring questions (mean $r = 0.52$) are no more linked than the average pair (mean $r = 0.53$), and the strongest pair is question 4 with question 6 ($r = 0.65$), which are not neighbors. So performance is broadly consistent across the whole exam (a student who does well tends to do well throughout), rather than a question-to-question chain. Even question 1 and question 6, the furthest apart, are moderately correlated ($r = 0.51$).

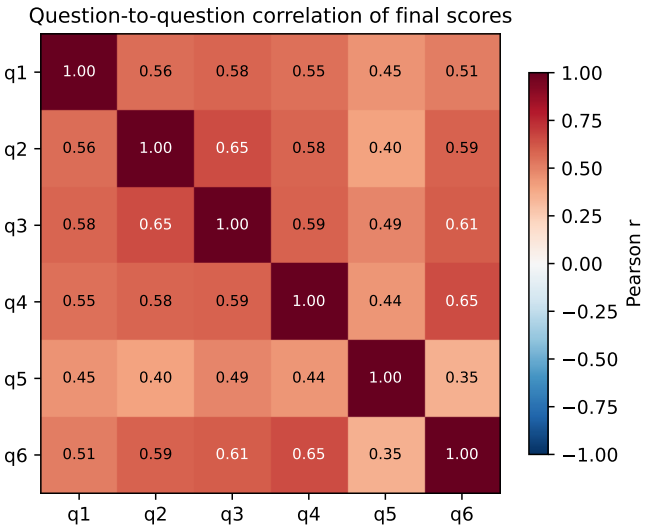

Figure 5.13: Correlation between the final scores on each pair of questions. All pairs are moderately-to-strongly and similarly correlated; neighboring questions are not special.

Does spending more time help? We measured, per question, the correlation between how long a student spent on it and their final score. The link is essentially absent on every question ($|r| \leq 0.22$). Spending longer on a question is not associated with a better score.

5.8 Conclusion

Grouping students by behavior, with the grade kept aside, gives four exam groups and three year groups. In our data the exam groups line up with the exam grade (means 1, 12, 40, 85), while the year groups differ far less (means 33, 35, 42), and the two groupings barely match each other. So, for these 569 students, the way a student worked in the exam went together with their exam result, while the way they worked during the year was only weakly linked to how they worked in the exam.

Two of the three research questions are answered here. **RQ1** asked whether students can be grouped into distinct, recognizable ways of working from their submission histories alone. They can, and the four exam groups differ in what they did, not only in what they scored. **RQ3** asked whether the way a student works during the year is related to the way they work in the exam. It is, but only weakly, so how a student worked during the year says little about how they worked in the exam. Section 6.5 adds a single-student view of the same result, and **RQ2** is the

subject of Chapter 6.

What these results do not show

We note five limitations.

- **The groups are not sharply separated.** The average silhouette is 0.185 for the exam and 0.164 for the year. Rousseeuw [24] calls an average of 0.28 "rather narrow, which indicates a relatively weak clustering structure". The groups are a useful way of describing how these students worked, not sharply separated kinds of student, which is what we expect from behavior that changes gradually from one student to the next.
- **A small group.** Exam group E1 has only 15 students. It is not a single outlier and it has a clear, extreme feature signature (large edits that leave the score unchanged, on code using very few concepts), so we keep it and read it by its features, but a group this size should be interpreted with care.
- **Significance is not effect size.** With 569 students the year-to-exam link is significant but small; we report the effect size ($V = 0.159$) next to the p-value and do not call the two groupings independent on the basis of a near-zero index alone.
- **The year signal is thin.** The year groups are less separated than the exam groups (average silhouette 0.164 against 0.185) and differ little in outcome (mean grades 33, 35 and 42, against 1 to 85). We treat the year structure as a weak split rather than a set of clearly separate kinds of student, and we checked it is not sensitive to the choice of k (below).
- **Grouping is at the profile level.** These groups describe a student's usual behavior, aggregated over questions; they are not single behaviors. The episode level is treated in Chapter 6.

Chapter 7 answers the three research questions from these results and states the threats to validity that apply to the whole study.

Chapter 6

Behavior at the Episode Level

Chapter 5 grouped students by their behavior at the level of a *profile*: one value per feature per student, then a group. This chapter turns to a finer level, the *episode*. It asks a simple question:

> **Beyond the scores they reach, do the behavior groups show different *ways of working* on the questions?**

Why go down to this level. Chapter 5 describes a student with one value per feature, summarized over all of their questions. A summary of that kind cannot show the order in which things happened, and the order is most of what "how a student works" means: two students can make the same number of attempts and end on the same score, and still have reached it by different routes. This chapter looks at the routes. That is what RQ2 asks for, since a behavior group is only worth having if it says something the grade does not, and the grade is a single number per student too.

What the chapter does. It sorts every episode into one of six shapes, and compares the exam with the year (Section 6.2) and the groups with each other (Section 6.3). It then reads 2 students from each of the 4 exam groups, 8 in all, one at a time, with their code and their submission traces (Section 6.4). Finally it follows those same students from the year into the exam (Section 6.5).

One thing limits what we can say about these sequences: the exam ran under a locked-down browser (Safe Exam Browser). A student could not run Python on their own machine, and a submission only told them whether the code ran, not the score and not whether it passed the hidden tests. Re-submitting was the normal way to test a change, so a long run of submissions is ordinary there and is not a sign of anything on its own.

This chapter **interprets** these sequences, it does not only list them. But submissions and code record what a student did, not what they meant by it. So

when we say why we think a student did something, we point to the submissions that led us to that idea. And when two explanations both fit, we give both, and we say which one we think fits better.

As in Chapter 5, a behavior group is defined by a **set of features**, not by the grade; **the grade appears next to each group only as a reference point**.

6.1 Two levels of behavior

The two levels were defined in Chapter 5, so we only recall them here: an **episode** is one student on one question (the ordered sequence of their submissions), and a **profile** is a student's usual way of working across questions, which is what the groups reflect. This chapter works at the episode level, where order matters: the same submissions in a different order are a different episode.

Reading behavior as the shape of a submission sequence, rather than as a final score, follows the earlier work reviewed in Chapter 2. One difference matters for the names used below. Perkins et al. [20] name what the student seems to be doing: a *stopper* gives up on a problem rather than explore it further, a *mover* tries one idea after another, and *tinkering*, a run of small repairs and retests, is a pattern they place inside mover behavior rather than beside it. Stopping and moving are themselves visible in our data: we can see that a student stopped submitting, or that they kept going. What we cannot see is why. Perkins et al. [20] define a *stopper* as a student who, faced with a difficulty, feels at a complete loss and is "unwilling to explore the problem any further". That unwillingness, not the stopping, is what the submissions do not show. So we name only what the sequence shows.

The two vocabularies can still be held side by side. An episode we call "few tries, no pass" leaves the record a stopper would leave. One we call "many tries, no pass" leaves the record of a mover who keeps going without getting there, tinkering included. We do not treat these as the same thing, and the reason is the method: Perkins et al. [20] sat with the students and asked them what they were doing, while we have only what they submitted. The same sequence can come from something else entirely, such as running out of time in the exam.

6.2 Episode shapes by setting

What we measure. We sort every episode into one of six shapes. The vocabulary is deliberately small and follows from three things we can read off a sequence: whether it reached the pass mark, how many submissions it took, and whether the score moved through one small decisive edit or through repeated changes. This gives six shapes, defined only by what we can observe (PASS means a score of at

least 50):

- **one-shot**: reached the pass mark within the first two submissions.
- **steady-climb**: reached the pass mark through gradual score rises.
- **breakthrough**: reached the pass mark, and at some point a *small* edit (at most 3 changed lines) was followed by the score rising by at least 50 points.[1]
- **pass after reversals**: reached the pass mark, but the score went down at least twice along the way.
- **many tries, no pass**: four or more submissions, the score never reached the pass mark.
- **few tries, no pass**: three or fewer submissions, the score never reached the pass mark.

The six descriptions can overlap: one episode can reach the pass mark through a small decisive edit and also drop twice on the way. We apply the rules in the order above and keep the first that matches, so every episode still ends up in exactly one shape. The order settles the clear cases first (a quick pass, or never reaching the pass mark); among the episodes that do reach the pass mark, a decisive small-edit jump is named before a path with several score drops, with a gradual climb as the default. For any set of episodes we then count how often each of the six shapes occurs. Those six percentages are what we call the **shape mix** of that set (of a setting, or later of a group).

What we expected. If the way a student works did not depend on the setting, the shape mix would look the same in the exam and during the year. If it does depend on the setting, the two mixes would differ.

What we find. They are very different (Figure 6.1), on the same 569 students. In the exam, 54% of episodes never reach the pass mark (42.5% "many tries, no pass" plus 11.6% "few tries, no pass"), and only 3.5% are one-shot. During the year, only about 4% of episodes never reach the pass mark, 30.5% are one-shot and 44.5% are breakthrough. So the way a student works is strongly tied to the setting, which is exactly why a single per-student label is a profile and not a behavior.

[1] This fixed three-line cut defines the episode *shape*. It is not the same cut as the "small fix, big gain" feature of Chapter 5, which calls an edit small when it is at or below the median edit size of the whole dataset (1 line in the exam, 2 in the coursework; Appendix A). The two look at the same idea (a small edit that pays off) at two different levels: one concrete episode here, a student's tendency there. The 50 is a rise, not a level reached, and it is absolute rather than relative for the reason given in Section 5.1: too many of these jumps start from a score of 0.

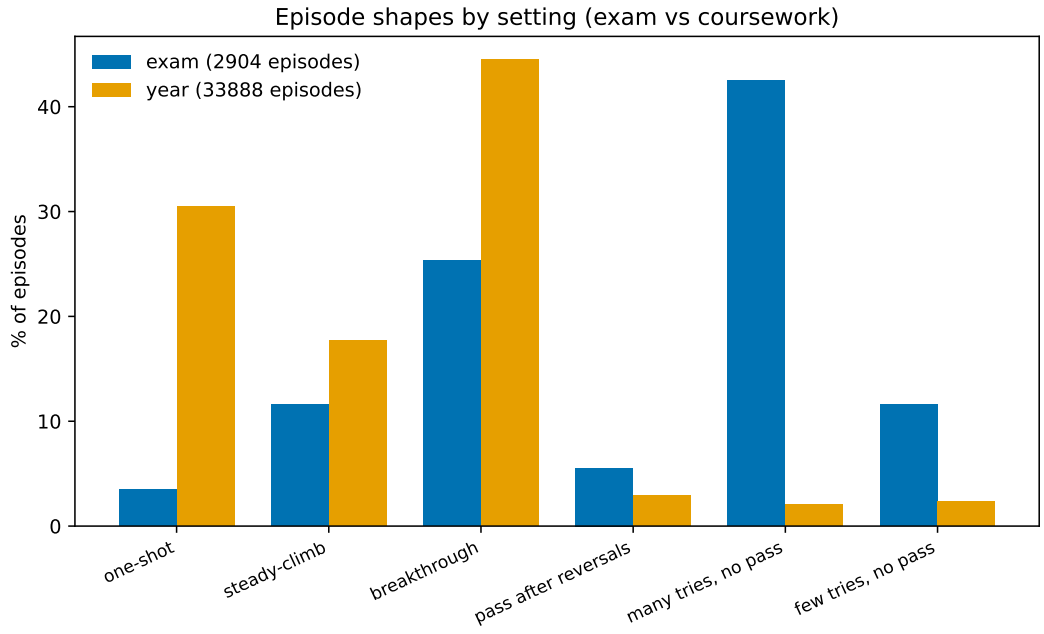

Figure 6.1: The mix of episode shapes in the exam and during the year, over the same 569 students.

6.3 Episode shapes by group

What we measure. For each group of Chapter 5 we take that group's students and look at the mix of episode shapes their questions fall into. This is the group's behavioral signature, in the same units as above.

What we expected. If the exam groups were only a re-labeling of the grade, they would differ in outcome but not in *how* the students worked. If they describe behavior, each group should have its own shape mix.

What we find (exam, Figure 6.2). Each group has a distinct signature:

- **E1** (grade mean 1, median 0): every episode is "no pass", split 77% "few tries" and 23% "many tries". Two ways of not passing live in the same group.
- **E2** (grade mean 12, median 4): mostly "many tries, no pass" (58%) with some "few tries, no pass" (28%); a small 7% breakthrough.
- **E3** (grade mean 40, median 33): a mix, 45% "many tries, no pass", 24% breakthrough, 13% steady-climb.
- **E4** (grade mean 85, median 90): 67% breakthrough, then steady-climb (11%) and one-shot (7%); "no pass" episodes are rare (8% together).

The exam groups were built partly from features that summarize these same shapes per student (how often an edit left the score unchanged, how often a small edit gave a large gain), so a matching shape mix is partly expected by construction. This figure is a **coherence check**: the per-student summaries reflect the episodes they are built from.

The evidence that the groups hold more than the score comes from two places that do not depend on how the groups were built: the exam-versus-year contrast above, and the case studies below, where two students in the same group with the same score work very differently. A first sign: E1 and E2 both score low, yet the grade cannot tell apart a student who stops after a few tries from one who submits many times without passing. The shape mix can.

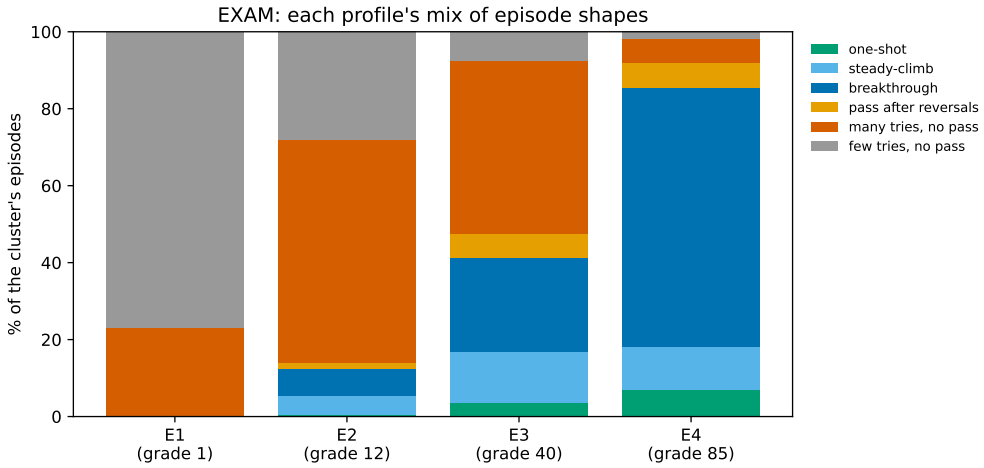

Figure 6.2: Each exam group as a mix of episode shapes. The mix is the group's signature.

What we find (year, Figure 6.3). The year groups differ far less. All three are dominated by "pass" shapes (one-shot and breakthrough), and "no pass" episodes are rare everywhere (about 2 to 4%). The groups differ mostly in the balance between one-shot and breakthrough (for example Y1 is 46% one-shot and 34% breakthrough, Y2 is 16% one-shot and 60% breakthrough), not in whether students reach the pass mark. This is the episode-level counterpart of the flat year grade from Chapter 5: the year groups are different ways of working that mostly all get there.

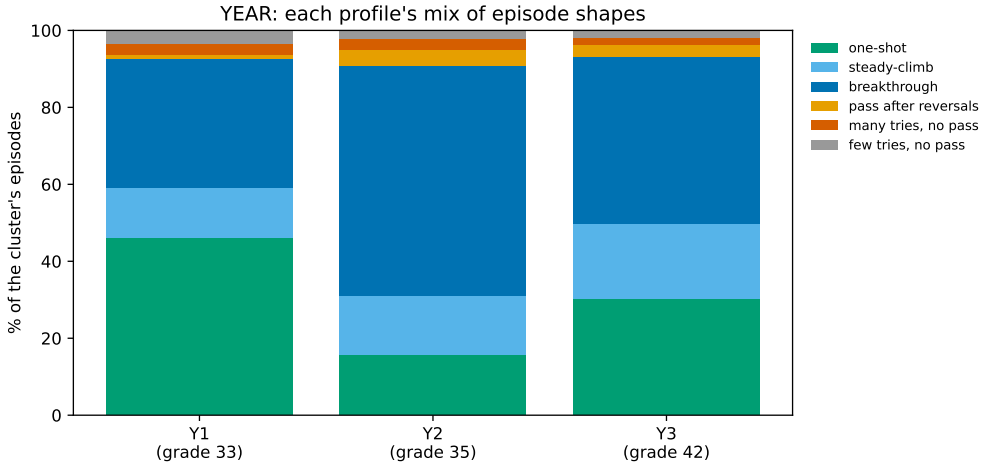

Figure 6.3: Each year group as a mix of episode shapes. The groups differ in the balance of "pass" shapes, not in whether the pass mark is reached.

Breakthrough and the question's pass rate

The breakthrough shape (a small edit followed by a large score rise) could in principle just mark questions with a high pass rate. We checked it (Figure 6.4). Reaching the pass mark is part of the shape, so some link with the pass rate is expected. Still, during the year, across 99 questions, the breakthrough share is essentially flat between the lower-pass-rate half (43.6%) and the higher-pass-rate half (42.8%), with almost no correlation ($r = 0.17$). In the exam there are only 6 questions and the share does track the pass rate ($r = 0.96$), but breakthroughs still make up about a fifth of episodes (22.7%) on the questions with the lowest pass rates. So the shape is not merely a by-product of questions with a high pass rate.

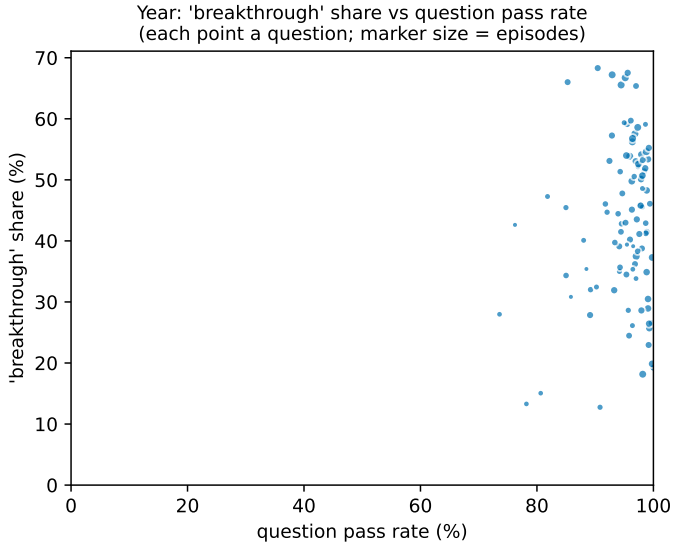

Figure 6.4: Year: the share of "breakthrough" episodes against the question's pass rate, one point per question.

6.4 Case studies: two students per exam group

How the cases are chosen. The group signatures above are averages; to see what they look like on real students, and to test the same-score, different-behavior claim directly, we read a few students' actual submission sequences. For each exam group we take the **medoid**, the student sitting closest to the center of the group in the feature space, as the most typical member; this is a fixed rule, not a hand-picked example. Because a group can hold more than one way of working, we also take a **contrasting** student from the same group, also near its center, whose most-iterated episode falls into a *different shape* from the medoid's (for example, in E1 the medoid keeps submitting without ever passing, while the contrasting student stops after a few tries). For each student we read their most-iterated question (chosen so there is an actual sequence to read, not because it is their typical question) and show the ordered trace: the score and the edit size at each submission. Two things follow from this: **First**, the medoid is typical in the *feature* space (its per-student medians sit at the group's center); it is not chosen to display the group's most common episode shape, and it need not. **Second**, because we read each student's *most-iterated* question, the episode we show is by construction one of that student's longer sequences, so a medoid can show a "many tries" episode even when most of its group's episodes are short. For each case, the trajectory is a numbered figure, and the code of a telling submission (and, where useful, the decisive edit) is shown

in a separate box just below it; these boxes are not numbered figures. A single student *illustrates* a group; it is not evidence about the whole group, which is what Chapter 5 provides.

We show E1, E4 and E3 in full below (trajectory figures and code); E2 more briefly, with its figures and code in Appendix C. All eight traces are reproducible.

6.4.1 Group E1: two paths to the same low result

Medoid. The medoid worked one question across 24 submissions and the score never left 0, with repeated edits throughout (Figure 6.5); its final code is Listing 6.1.

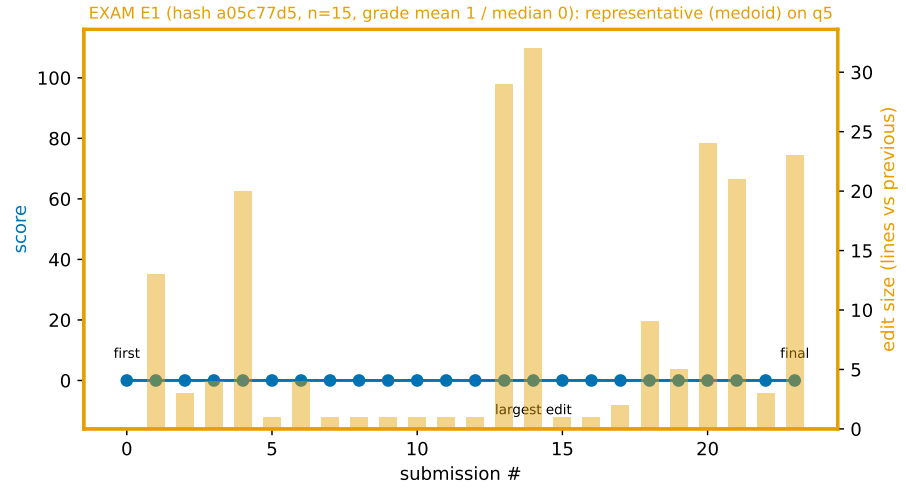

Figure 6.5: E1 medoid: 24 submissions on one question, the score stays at 0 throughout (score, left axis; edit size per submission, right axis).

Listing 6.1: E1 medoid (hash `a05c77d5`), last of 24 submissions on q5 (score 0). The code is valid Python (an `Employe` class with a `salaire` method), but the question also asked for a `Commercial` subclass, which is absent; the submission scores 0, as did all 24.

```
class Employe:

    def __init__(self, nom, taux_horaire, heures_travaillees):
        """ Initialise un employe
        pre: 'nom' est une chaine de caracteres
             'taux_horaire' est un float positif
             'heures_travaillees' est un entier positif
        post: initialise un employe avec 'taux_horaire' et 'heures_travaillees'
        """
        self.nom = nom
        self.taux_horaire = taux_horaire
        self.heures_travaillees = heures_travaillees
```

CONTINUES ↓

↑ CONTINUED

```
    def salaire(self):
        """ Calcule le salaire de l'employe
        pre:  -
        post: retourne le salaire de cet employe calcule comme le
              produit du taux_horaire et des heures_travaillees
        """
        return self.taux_horaire * self.heures_travaillees
```

Its largest single edit rewrote the whole class, and the score still did not move (Listing 6.2).

Listing 6.2: E1 medoid, largest edit (submission 13 to 14). A 32-line rewrite replaces the real constructor assignments with hardcoded values and mangles the methods; the score stays at 0, as it did through all 24 submissions. Removed lines in red, added lines in green.

```
     def __init__(self, nom, taux_horaire, heures_travaillees):
-        self.nom = nom
-        self.taux_horaire = taux_horaire
-        self.heures_travaillees = heures_travaillees

+        self.nom = bob
+        self.taux_horaire = 20
+        self.heures_travaillees = 160
     def salaire(self):
-        return self.taux_horaire * self.heures_travaillees
+        if self.taux_horaire >= 0 :
+
+        if  self.heures_travaillees(n , n + 1):
+
+
+
+
+
+        return self.taux_horaire * self.heures_travaillees
+
+    def  bonus(self):
+        ef __init__(self, nom,  produits_vendus, bonus_par_produit):
+
+        self.nom = bob
+        self. produits_vendus= 156
+        self.bonus_par_produit = 2
+
+    return self. produits_vendus *  self.bonus_par_produit
+
+
```

By the last submission the student had cleaned this up again: the final code above is valid Python but still missing the `Commercial` subclass the question asked for.

Contrasting student. The contrasting student made only 3 submissions on their most-iterated question and also finished at 0 (Figure 6.6); their final code is

in Appendix C.

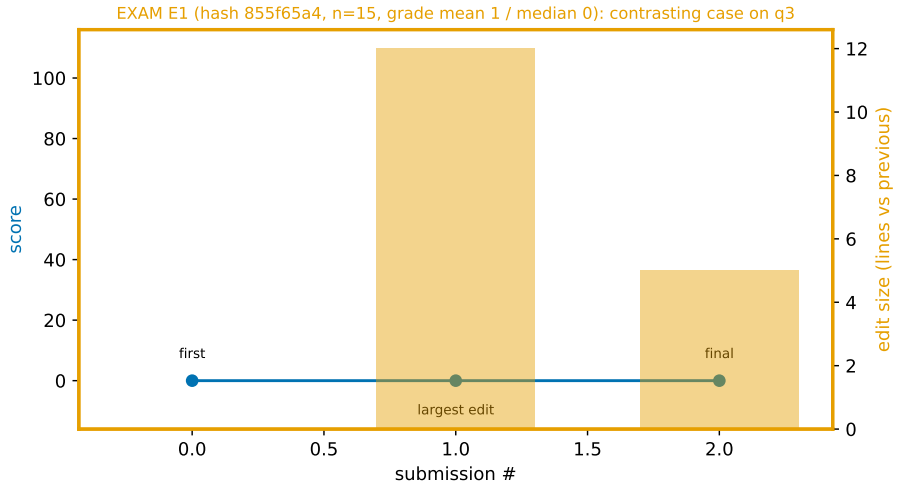

Figure 6.6: E1 contrasting student: 3 submissions, the score stays at 0. Same group and same result as the medoid, reached in 3 submissions instead of 24.

What this shows. Two competing explanations fit the group: *(H1)* few submissions and little code before stopping; *(H2)* many submissions without ever reaching the pass mark. H2 fits the medoid's trace better, H1 the contrasting student's. The grade (0 for both) cannot separate these; the sequence can. Under the exam conditions recalled above, the repeated submissions of H2 are consistent with a student using submission as the only way to run their code, but we cannot confirm that intent from the trace.

6.4.2 Group E4: two paths to a high score

Medoid. The medoid reached 90 in 14 submissions: the score sat at 0 for the first few tries, then a run of small edits took it $0 \to 5 \to 47 \to 85 \to 90$ (Figure 6.7).

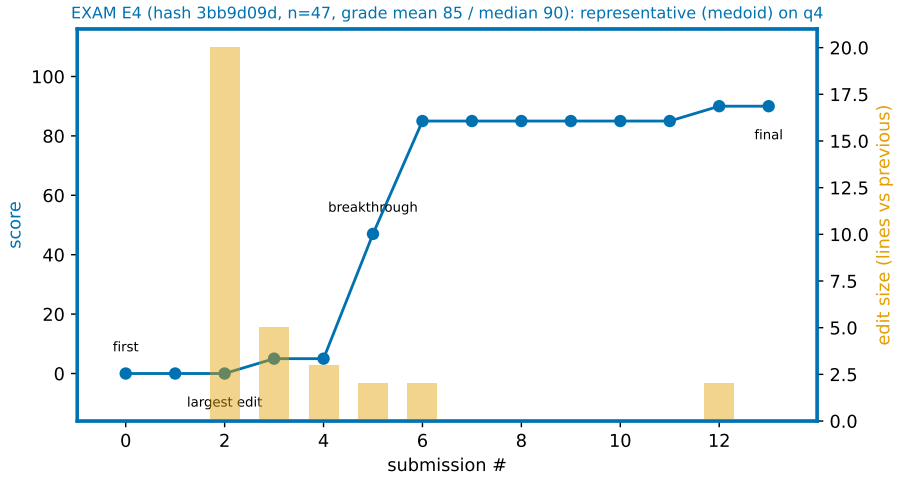

Figure 6.7: E4 medoid: the score sits at 0 for the first few submissions, then a run of small edits takes it to 90 over 14 submissions.

Listing 6.3: E4 medoid (hash `3bb9d09d`), last of 14 submissions on q4 (score 90). A near-complete solution; one case is not handled to specification (the missing-file case returns a string instead of raising `FileNotFoundError`). The docstring is omitted for space.

```
def longest_word(nom_fichier):
    try :
        mot = ""
        ligne = 0
        n_ieme = 0
        with open(nom_fichier,"r") as f :
            ligne_prov = -1
            for line in f:
                ligne_prov += 1
                chaine=line.strip()
                if not chaine:
                    continue
                numero = -1
                l = chaine.split(" ")
                for word in l :
                    if word.strip().isalpha():
                        numero += 1
                    if len(word.strip()) > len(mot) :
                        mot = word
                        ligne = ligne_prov
                        n_ieme = numero
        return (ligne, n_ieme, mot)

    except FileNotFoundError :
        return "FileNotFoundError : Le fichier n'existe pas."
```

Its decisive edit was small but not cosmetic. The variable `ligne_prov` had been written `line_prov` on two lines; that name is never defined, so the code failed

wherever it was used. Correcting the spelling let the function run, and the score rose from 5 to 47 (below). This is the "small edit, big change" idea in one concrete case: a one-character slip was holding the score down.

Listing 6.4: E4 medoid, decisive edit (submission 4 to 5). A mistyped variable (`line_prov` instead of the defined `ligne_prov`) is corrected on two lines; the function then runs and the score rises from 5 to 47. Removed lines in red, added lines in green.

```
                    continue
                numero = -1
-               line_prov += 1
+               ligne_prov += 1
                l = chaine.split(" ")
                for word in l :
                    if len(word.strip()) > len(mot) :
                        mot = word
-                       ligne = line_prov
+                       ligne = ligne_prov
                        n_ieme = numero
        return (ligne, n_ieme, mot)
```

Contrasting student. The contrasting student reached the same 90, but over 35 submissions with the score rising and falling many times before it settled (Figure 6.8).

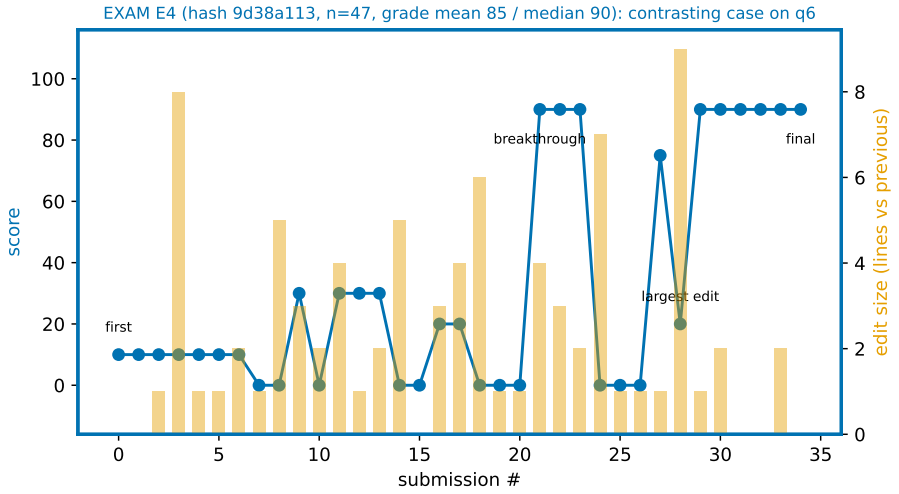

Figure 6.8: E4 contrasting student: the same final score 90, reached over 35 submissions with the score rising and falling several times first.

Listing 6.5: E4 contrast (hash `9d38a113`), last of 35 submissions on q6 (score 90).

```
def moyenne_ponderee(notes_etudiant, credits_cours):
    current_notes = notes_etudiant.head
    current_credit = credits_cours.head
    moy = 0
    div = 0
    if credits_cours.length > notes_etudiant.length:
        while current_credit != None:

            if current_credit.cargo[0] == current_notes.cargo[0]:

                moy += current_notes.cargo[1] * current_credit.cargo[1]
                div += current_credit.cargo[1]
                if current_notes.next != None:
                    current_notes = current_notes.next
            current_credit = current_credit.next
    else:
        while current_notes != None:

            if current_credit.cargo[0] == current_notes.cargo[0]:

                moy += current_notes.cargo[1] * current_credit.cargo[1]
                div += current_credit.cargo[1]
                if current_credit.next != None:
                    current_credit = current_credit.next
            current_notes = current_notes.next
    return moy / div
```

The score did not climb straight. At one point a working loop was wrapped in a malformed condition, dropping the score to 0 before the student restored it (below).

Listing 6.6: E4 contrast, one of several reversals (submission 23 to 24). A working loop is wrapped in a malformed `if` (no colon), which breaks it; the score falls from 90 to 0, and is recovered later. Removed lines in red, added lines in green.

```
     moy = 0
     div = 0
-    while current_credit != None:
+    if credits_cours.length > notes_etudiant.length
+        while current_credit != None:

-        if current_credit.cargo[0] == current_notes.cargo[0]:
+            if current_credit.cargo[0] == current_notes.cargo[0]:

-            moy += current_notes.cargo[1] * current_credit.cargo[1]
-            div += current_credit.cargo[1]
-            if current_notes.next != None:
+                moy += current_notes.cargo[1] * current_credit.cargo[1]
+                div += current_credit.cargo[1]
+                if current_notes.next != None:
                     current_notes = current_notes.next
```

CONTINUES ↓

↑ CONTINUED

```
-         current_credit = current_credit.next
+             current_credit = current_credit.next
```

What this shows. Within one high-scoring group the sequences still differ: a fairly direct climb for the medoid, a much longer path with many reversals for the contrast. Neither one had the answer from the start; both worked it out by submitting.

6.4.3 Group E3: same group, two outcomes

Medoid. The medoid made 37 submissions on q5 and settled at a partial score (20); its trajectory is Figure 6.9 and its final code Listing 6.7.

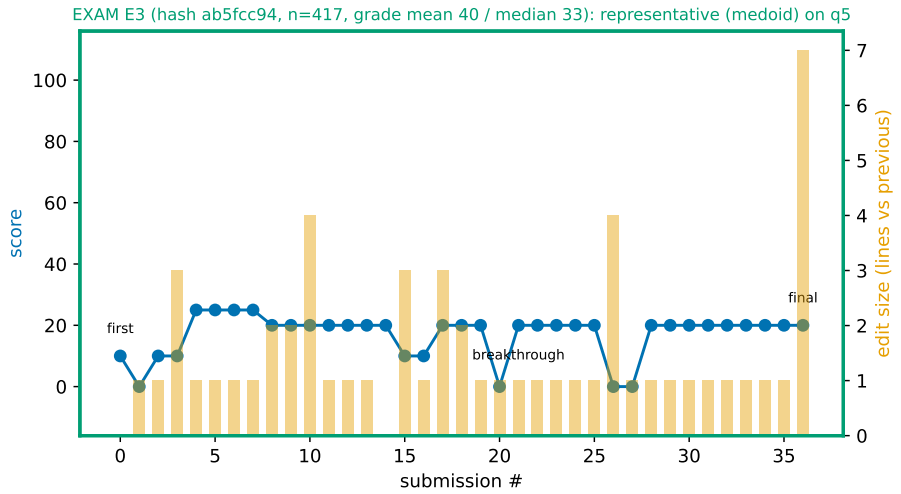

Figure 6.9: E3 medoid: 37 submissions on q5, the score settling at a partial value (20) (score, left axis; edit size per submission, right axis).

Listing 6.7: E3 medoid (hash `ab5fcc94`), last of 37 submissions on q5 (score 20). A Commercial subclass with a bonus method and a redefined salaire, but with malformed method definitions; the score settled at 20.

```
class Commercial(Employe) :
    def __init__(self, nom, taux_horaire, heures_travaillees, produits_vendus,
     bonus_par_produit ):
        self.nom = nom
        self.taux_horaire = taux_horaire
        self.heures_travaillees = heures_travaillees
        self.produits_vendus = produits_vendus
        self.bonus_par_produit = bonus_par_produit
    def __bonus__(self):
        return self.produits_vendus * self.bonus_par_produit
```

CONTINUES ↓

↑ CONTINUED

```
    def salaire(self, ):
        salire = super().salaire() + __bonus__(self)
        return salaire
    def __str(self):
        return salaire
```

Its decisive edit went the wrong way (Listing 6.8).

Listing 6.8: E3 medoid, decisive edit (submission 25 to 26). Two methods are rewritten into invalid syntax (`return =`), which breaks the class; the score falls from 20 to 0, and later returns to 20. Removed lines in red, added lines in green.

```
        self.bonus_par_produit = bonus_par_produit
    def __bonus__(self):
-       prime = self.produits_vendus * self.bonus_par_produit
-       return prime
+       return = self.produits_vendus * self.bonus_par_produit
+
    def __salaire__(self, ):
-       nsalaire = super().salaire() + __bonus__(self)
-       return nsalaire
+       return= super().salaire() + __bonus__(self)
+
    def __str(self):
        return nsalaire
```

Contrasting student. A student in the *same* group reached full marks (100) on the same question over 23 submissions (Figure 6.10). A single line did it: the parent constructor `Employe.__init__` had been called with no arguments, so the object was built without its name, hourly rate or hours worked; passing those three values fixed the object and the score went from 10 to 100 (below).

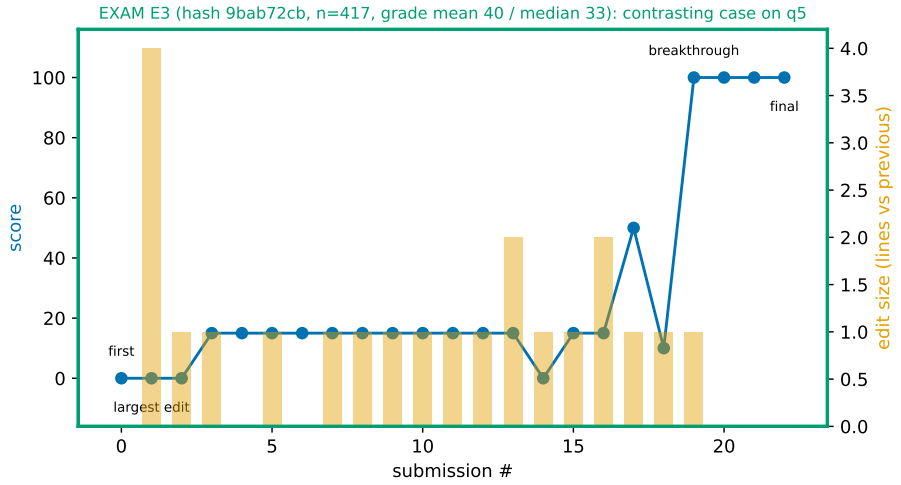

Figure 6.10: E3 contrasting student: 23 submissions on q5, reaching full marks (100).

Listing 6.9: E3 contrast (hash `9bab72cb`), last of 23 submissions on q5 (score 100). A complete `Commercial` subclass.

```
class Commercial(Employe):
    def __init__(self, nom, taux_horaire, heures_travaillees, produits_vendus,
     bonus_par_produit):
        Employe.__init__(self, nom, taux_horaire, heures_travaillees)
        self.produits_vendus = produits_vendus
        self.bonus_par_produit = bonus_par_produit


    def bonus(self):
        return self.produits_vendus * self.bonus_par_produit


    def salaire(self):
        salaire = self.bonus() + Employe.salaire(self)
        return salaire
```

Listing 6.10: E3 contrasting student (hash `9bab72cb`), decisive edit (submission 18 to 19). The parent constructor is now called with its three arguments instead of none, so the object is built correctly; the score rises from 10 to 100. Removed line in red, added line in green.

```
 class Commercial(Employe):
     def __init__(self, nom, taux_horaire, heures_travaillees, produits_vendus,
     bonus_par_produit):
-        Employe.__init__(self)
+        Employe.__init__(self, nom, taux_horaire, heures_travaillees)
         self.produits_vendus = produits_vendus
         self.bonus_par_produit = bonus_par_produit
```

What this shows. Same group, opposite outcomes: the medoid's many edits settle at 20, while the contrasting student's edits convert into 100. The grade would place these two far apart, yet they sit in the same behavior group, and the sequences show why: both iterate on the same question, but one lands the fix and the other does not.

6.4.4 Group E2: little code, little net change

Medoid. The medoid submitted 68 times on q1 and settled at a low partial score (5). **Contrasting student.** The contrasting student made 30 submissions on q2 and finished at 0. Both ran long sequences that moved the score very little, which is what E2's low share of tries that improved describes at the group level. Their trajectories and code are in Appendix C (Figures C.1 and C.2).

So one feature signature covers more than one sequence, even inside a single group. This is why we treat a group as a tendency and show real traces beside it, instead of naming it after one student's story.

6.5 The same student across settings

What we measure. The case studies above are different students in one setting each. Behavior as an evolution is a property of *one* student across settings. So we take the same eight students (each group's medoid and contrasting student) and follow them back into the year: which year group they fall in, beside their exam group (Table 6.1). For the clearest single case we also put the student's year episode-shape mix beside their exam one (Figure 6.11).

Table 6.1: The eight case students across settings: their exam group, that student's overall exam grade (all six questions), and the year group they fall in.

Exam group	Case	Exam grade (all 6 q)	Year group
E1	medoid	0	Y1
E1	contrast	2	Y1
E2	medoid	3	Y1
E2	contrast	2	Y3
E3	medoid	25	Y1
E3	contrast	56	Y3
E4	medoid	97	Y1
E4	contrast	95	Y3

What we find. All four exam medoids fall in the *same* year group, **Y1**, the group that did the least coursework during the year (fewest questions attempted,

fewest active weeks; Chapter 5). The four contrasting students are more spread: the E1 contrast is also in Y1, but the E2, E3 and E4 contrasts are all in **Y3**, the group that attempted the most questions. The sharpest case is E4 (Figure 6.11): its medoid and its contrasting student both reach the top exam group, with almost the same exam grade (97 and 95), yet the medoid sat in Y1 (did little coursework, and passed most of the few year questions they did in one shot or through a breakthrough) while the contrast sat in Y3 (attempted the most questions all year). The same exam outcome, reached from opposite year behavior. So a student's exam group does not fix their year group: even two students from the *same* exam group can sit in different year groups.

This is the single-student version of the weak group-to-group link in Chapter 5 (Cramér's $V = 0.159$). It is only eight students, so we read it as an illustration, not a rule; the real strength of the year-to-exam link is the number in Chapter 5.

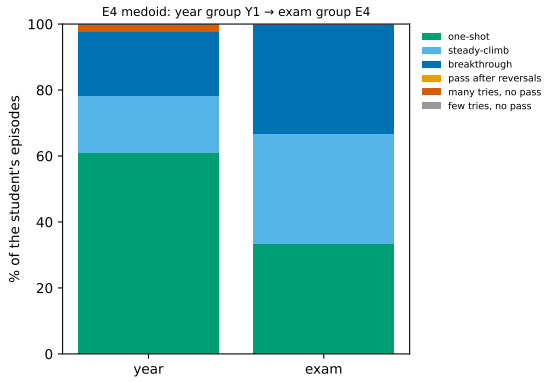

Figure 6.11: The E4 medoid across settings: mostly "pass" shapes during the year (year group Y1), a mixed set of episodes in the exam (exam group E4).

The same year-versus-exam picture for the other seven case students (the E1, E2 and E3 medoids, and all four contrasting students) is in Appendix C (Figures C.3 and C.4); the medoids all sit in year group Y1, the contrasting students mostly in Y3.

6.6 Conclusion

Reading behavior at the episode level answers the chapter's question: beyond the scores they reach, the exam groups show different *ways of working*. Each exam group has its own mix of episode shapes, from E1 (never reaches the pass mark) to E4 (mostly reaches it through a small decisive edit), and the case traces show that

students with the same group and even the same score can follow very different sequences. The year groups differ far less, which matches their flat grade.

This answers **RQ2**: the groups do differ in how students work through a task, beyond the score they reach, since two students with the same score can arrive there by different sequences. Section 6.5 also adds to **RQ3**: followed one at a time, the same students work differently in the year and in the exam, which is the single-student form of the weak link measured in Chapter 5.

What these results do not show

We note five limitations.

- **Intent is inferred, not observed.** Nothing in a submission record states why a student did what they did. Where two explanations fit we give both and say which one fits better, but the better fit is still an argument from the submissions.
- **Few episodes per student in the exam.** With six questions, a student has at most six exam episodes, about five on average and fewer for those who did not reach the later questions, so one student's exam mix is noisy. The mixes computed over all 569 students together, and the year mixes (dozens of episodes per student), are far more stable. More exam episodes for these same students would have to come from a second exam later in the year, taken by those who did not pass in January, or from an earlier one, taken by those who had already sat the course before.
- **The cases illustrate, they do not prove.** A medoid and a contrasting student per group are examples that help reading; they are not evidence about the whole group, which is what Chapter 5 is for.
- **Shapes are defined on the sequence, not on intent.** "breakthrough" or "many tries, no pass" describe what the submissions show, nothing more; and the breakthrough shape is not fully separated from a question's pass rate (Figure 6.4).
- **How common "breakthrough" is depends on where we cut it.** At a rise of 25 points the shape covers 34.7% of exam episodes and 51.1% of year ones, at 50 points 25.3% and 44.5%, at 75 points 15.8% and 33.7%. The gap between the two settings holds at every cut, so the comparison does not rest on the choice, but the percentages themselves should be read with the cut in mind.

Chapter 7 answers the three research questions together and states the threats to validity that apply to the whole study.

Chapter 7

Conclusion

This thesis asked what the record of a student's attempts shows once the final score is set aside. The data came from an introductory Python course at UCLouvain and its INGInious platform, and covered the 569 students present in both the coursework term and the January exam. This chapter answers the three research questions, states what the study cannot claim, and points to the work that would follow.

7.1 Answer to RQ1: can students be grouped into recognizable ways of working?

Yes, and the groups hold together well enough to be described, but they are not sharp. Using behavior features only, with the grade kept out of the grouping, the exam splits into **four groups** ($n = 15$, 90, 417, 47) and the coursework term into **three** ($n = 155$, 93, 321).

Which ways of working these are. Each group has its own feature signature. **E1** makes few attempts, but each edit is large and almost none of them raise the score; the code that results uses very few programming concepts. **E2** writes little code, makes small edits, and rarely improves the score from one try to the next. **E3** holds 417 of the 569 students and stands out on nothing: it is close to the average on every feature. **E4** is defined by the small edit that produces a large gain, attempts more questions, and more of its tries raise the score. The number of groups was taken from four measures rather than one, and the choice was made on which cut survives resampling.

Two qualifications belong with that answer. The separation between the groups is weak: the average silhouette is 0.185 for the exam and 0.164 for the coursework. The groups describe tendencies, not distinct kinds of student. That is what we should expect, **since behavior changes gradually from one student to the**

next. And the coursework structure is thinner than the exam one, with groups that differ far less in outcome (mean exam grades of 33, 35 and 42, against 1 to 85 for the exam groups).

7.2 Answer to RQ2: do the groups differ in how students work through a task, beyond the score?

Yes. Sorting every sequence of submissions to one question into one of six observational shapes gives each group a different mix.

How they differ. In the exam, every episode of E1 ends without reaching the pass mark, split between 77% "few tries" and 23% "many tries": two different ways of not passing sit inside one group, which a score cannot separate. E2 is mostly "many tries, no pass" (58%). E3 is mixed (45% "many tries, no pass", 24% breakthrough). E4 is dominated by breakthrough (67%), with "no pass" episodes rare (8% together).

The same reading shows that the way a student works depends heavily on the setting. In the exam, 54% of episodes never reach the pass mark and only 3.5% are one-shot. During the term, about 4% never reach it, 30.5% are one-shot and 44.5% pass through a single decisive edit. The individual case studies make the same point at the level of one student: two students who end with the same exam outcome can arrive there by opposite routes.

7.3 Answer to RQ3: is the way a student works during the year related to the way they work in the exam?

Only weakly. The two groupings agree barely more than chance, so knowing a student's year group tells us little about which exam group they fall in. No predictive model was fitted here: the measures below compare two groupings of the same students, they do not measure how well one forecasts the other. The link between the two groupings is statistically significant but small ($\chi^2(6) = 28.6$, $p < 0.001$, Cramér's $V = 0.159$), and the two groupings agree barely more than chance would (adjusted Rand index 0.07). Students from the year group that attempted the fewest questions are somewhat more likely to fall in the lower exam groups (29% against a base rate of 18%), but every year group still sends most of its students to the same large middle exam group. The result does not depend on

how finely the coursework is split: across two, three and four year groups, Cramér's V stays between 0.13 and 0.21 with the adjusted Rand index near zero.

The case studies point the same way. All four exam medoids, from the group with a mean grade of 1 to the group with a mean of 85, fall into the same year group. A student's way of working in the exam is not fixed by their way of working during the term.

This is a negative result, and it is reported as one. It is also in line with what this area produces: a behavior measure built on compilation events explained 11% and 25% of the variance in two attainment measures [12], and lost its relationship with marks entirely when recomputed in another course [11].

7.4 What a teacher can take from this

We did not set out to give teaching advice, and one course cannot support much of it. Still, four things follow from what we found, and the course already holds the data behind each of them.

A low mark is not one thing. E1 and E2 both score near zero, but they get there differently: E1 makes few but large edits that almost never raise the score, and about a third of its final submissions do not even run, while E2 writes little and rarely improves from one try to the next. A teacher who sees only the marks treats these students the same. The submission record separates them, and it does so during the term rather than after the exam.

Code that never runs is visible without grading it. About a third of E1's final submissions had a syntax error, and 3 of its 15 students never produced a single submission that parsed. A platform can raise that signal on its own.

How a student works during the year says little about how they will work in the exam. The link between the two groupings is weak ($V = 0.159$). Using coursework behavior to anticipate exam behavior is not supported here, and the two settings differ far more than the students do: 54% of exam episodes never reach the pass mark, against about 4% during the term.

More time is not better work. How long a student spent on a question is essentially unrelated to their score ($|r| \leq 0.22$).

7.5 Threats to validity

One course, one institution, one term. All the data comes from a single Python course and a single exam session. Ihantola et al. [11] report that 81% of the studies they reviewed have that same shape, and ask researchers to refrain from drawing strong conclusions from them. The groups reported here should be read

as a description of these 569 students, not as a typology of novice programmers. We also see each student sit the exam once. Bachelor 1 students who did not pass in January take it again in June or in August, and how the same student works at that second sitting, after a term of coursework and one failed attempt behind them, would be worth seeing. The data allows it to be tracked; we did not do it here.

What a submission record cannot show. We only see the moments a student pressed submit. Ihantola et al. [11] name the cost: submissions are far apart, and nothing that happens between two of them is recorded. Everything a student did in between, including reading their own code, is invisible here. The one thing Perkins et al. [20] call decisive, following your own code closely, cannot be seen in this data at all.

Work done outside the platform. During the term a student can write and test code locally and submit only near-final work, so the coursework record may understate the number of attempts. This does not apply to the exam, which runs under a locked-down browser.

The coursework we study is practice work only. Each mission week also holds a graded assignment with a deadline, the *phase de réalisation*, where consistent work earns extra points on the exam. That part is not in our data, so the coursework side of every comparison here rests on questions with no deadline and no mark that counts.

Nothing is observed about intent. The six shapes and the group signatures describe sequences of attempts. They do not show why a student stopped, whether they understood their own solution, or what they were trying to do. Any statement of that kind in this document is marked as a hypothesis.

The group-to-shape check is a coherence check, not independent validation. The groups were formed partly from aggregate features that are related to the episode shapes, so some agreement between them is expected. It shows that the per-student summary reflects the episodes it is built from; it does not confirm the groups against an outside criterion.

A small group and an uneven shape. Exam group E1 holds only 15 students. It is kept because it has a clear and extreme feature signature rather than because it is large, and it should be read with care. Separately, the breakthrough shape does not control for how hard a question is, so an easy coursework question inflates it.

7.6 Future work

Model the order of events explicitly. At the moment a student's whole run of submissions on one question gets a single label, such as "many tries, no pass". What happened, and in what order, inside that run is thrown away. Process mining

keeps the order: it takes sequences of labeled steps and draws the paths students most often follow, and Ardimento et al. [2] have used it on novice coding data. To do the same here, each submission would first need a label saying what happened to it, such as "did not run", "ran but failed the tests" or "passed". Our records hold the score of each submission but not what went wrong, so those labels would have to be built first.

Ask the students. Nothing here reaches understanding or intent, and that boundary is structural rather than a matter of more data. Think-aloud sessions [14] or concept inventories [9] would reach it, and Lokkila [17] states that his own groups need exactly this kind of qualitative check.

Repeat the comparison elsewhere. The coursework-to-exam comparison is the part of this work with no precedent in the literature reviewed in Chapter 2. Whether the link stays weak in another course, another language or another exam format is an open question, and the reproduction reported by Ihantola et al. [11] suggests the answer may well differ by context.

Follow the same students into their later programming courses. This study pairs one term with the exam that closes it, so it sees each student over a few months. Several later courses in the same degree program are hosted on the same platform, which means the same students keep producing comparable records as they gain experience. Following them forward would show whether a student's way of working changes over that time, which a single term cannot say. This is a different question from re-examining students who repeat the course, whose second attempt is shaped by having already seen the material.

References

[1] Arif Altun and Sacide Guzin Mazman. "Identifying Latent Patterns in Undergraduate Students' Programming Profiles". In: *Smart Learning Environments* 2 (2015), p. 13. DOI: 10.1186/s40561-015-0020-0.

[2] Pasquale Ardimento et al. "Understanding Coding Behavior: An Incremental Process Mining Approach". In: *Electronics* 11.3 (2022), p. 389. DOI: 10.3390/electronics11030389.

[3] Ryan S. Baker and Paul Salvador Inventado. "Educational Data Mining and Learning Analytics". In: *Learning Analytics: From Research to Practice*. Ed. by J. A. Larusson and B. White. New York: Springer, 2014. Chap. 4, pp. 61–75. DOI: 10.1007/978-1-4614-3305-7_4.

[4] Tadeusz Caliński and Jerzy Harabasz. "A dendrite method for cluster analysis". In: *Communications in Statistics* 3.1 (1974), pp. 1–27. DOI: 10.1080/03610927408827101.

[5] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd. Lawrence Erlbaum Associates, 1988.

[6] Harald Cramér. *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[7] David L. Davies and Donald W. Bouldin. "A cluster separation measure". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-1.2 (1979), pp. 224–227. DOI: 10.1109/TPAMI.1979.4766909.

[8] James Fraser. "Learning to Code: An Examination of Novice Programmer Profiles". PhD thesis. Guelph, Ontario, Canada: University of Guelph, 2023.

[9] Julie Henry. "Évaluer la compréhension en informatique à partir des représentations erronées". PhD thesis. Namur, Belgium: Université de Namur, 2022. ISBN: 978-2-39029-163-3.

[10] Lawrence Hubert and Phipps Arabie. "Comparing partitions". In: *Journal of Classification* 2.1 (1985), pp. 193–218. DOI: 10.1007/BF01908075.

[11] Petri Ihantola et al. “Educational Data Mining and Learning Analytics in Programming: Literature Review and Case Studies”. In: *Proceedings of the 2015 ITiCSE on Working Group Reports (ITiCSE-WGR ’15)*. ACM, 2015, pp. 41–63. DOI: 10.1145/2858796.2858798.

[12] Matthew C. Jadud. “Methods and Tools for Exploring Novice Compilation Behaviour”. In: *Proceedings of the Second International Workshop on Computing Education Research (ICER)*. 2006, pp. 73–84. DOI: 10.1145/1151588.1151600.

[13] Leonard Kaufman and Peter J. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, 1990.

[14] Cazembe Kennedy and Eileen T. Kraemer. “Qualitative Observations of Student Reasoning: Coding in the Wild”. In: *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education*. ITiCSE ’19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 224–230. DOI: 10.1145/3304221.3319751.

[15] Essi Lahtinen, Kirsti Ala-Mutka, and Hannu-Matti Järvinen. “A Study of the Difficulties of Novice Programmers”. In: *Proceedings of the 10th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE ’05)*. ACM, 2005, pp. 14–18. DOI: 10.1145/1067445.1067453.

[16] Qianhui Liu and Luc Paquette. “Using Submission Log Data to Investigate Novice Programmers’ Employment of Debugging Strategies”. In: *LAK23: 13th International Learning Analytics and Knowledge Conference*. New York, NY, USA: ACM, 2023, pp. 637–643. DOI: 10.1145/3576050.3576094.

[17] Erno Lokkila. “Understanding Novice Programmer Behavior on Introductory Courses: Learning Analytics Approach”. PhD thesis. Turku, Finland: University of Turku, 2023. ISBN: 978-951-29-9148-8.

[18] Sonsoles López-Pernas et al. “Can AI deliver appropriate support for diverse student profiles?: A large-scale evaluation”. In: *Computers in Human Behavior: Artificial Humans* 9 (2026), p. 100357. DOI: 10.1016/j.chbah.2026.100357.

[19] Gabriel Silva de Oliveira et al. “Exploring Novice Programmer Testing Behavior: A First Step to Define Coding Struggle”. In: *Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE 2024)*. ACM, 2024, pp. 1251–1257. DOI: 10.1145/3626252.3630851.

[20] D. N. Perkins et al. “Conditions of Learning in Novice Programmers”. In: *Journal of Educational Computing Research* 2.1 (1986), pp. 37–55. DOI: 10.2190/GUJT-JCBJ-Q6QU-Q9PL.

[21] Chris Piech et al. “Deep Knowledge Tracing”. In: *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*. Preprint: arXiv:1506.05908. 2015, pp. 505–513.

[22] Anthony Robins, Janet Rountree, and Nathan Rountree. “Learning and Teaching Programming: A Review and Discussion”. In: *Computer Science Education* 13.2 (2003), pp. 137–172. DOI: 10.1076/csed.13.2.137.14200.

[23] Cristóbal Romero and Sebastián Ventura. “Educational Data Mining: A Review of the State of the Art”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C* 40.6 (2010), pp. 601–618. DOI: 10.1109/TSMCC.2010.2053532.

[24] Peter J. Rousseeuw. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis”. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

[25] Joe H. Ward. “Hierarchical Grouping to Optimize an Objective Function”. In: *Journal of the American Statistical Association* 58.301 (1963), pp. 236–244. DOI: 10.1080/01621459.1963.10500845.

[26] Christopher Watson and Frederick W. B. Li. “Failure Rates in Introductory Programming Revisited”. In: *Proceedings of the 2014 Conference on Innovation and Technology in Computer Science Education (ITiCSE ’14)*. ACM, 2014, pp. 39–44. DOI: 10.1145/2591708.2591749.

[27] Barry J. Zimmerman. “Becoming a Self-Regulated Learner: An Overview”. In: *Theory Into Practice* 41.2 (2002), pp. 64–70. DOI: 10.1207/s15430421tip4102_2.

Appendix A

Reproducibility Appendix

Pipeline

The analysis runs in this order: extract the submissions → clean and audit the data → build the per-student features → clean the feature set (remove near-constant and redundant features) → group students (hierarchical clustering) → describe and compare the groups. Exam scores are the full official INGInious re-grade of every submission.
The grouping step, in enough detail to repeat it: four right-skewed features (attempts per question, gap between tries, lines changed per edit, code lines) go through $\log(1 + x)$, all features are z-scored, then Ward linkage on the Euclidean distance, cut at the chosen k. Stability is the mean adjusted Rand index over **50 draws of a random** 80% **subsample**, each re-clustered and compared with the full-data grouping, seed fixed.

How each threshold is derived

Table A.1 collects every cut-off used in this work. No cut-off is a hand-picked round number: except for the course pass mark, each is computed from this dataset's own distribution, separately for the exam and the year. Let a *gap* be the number of seconds between two consecutive submissions of the same question (gaps over 24 hours are dropped, as they mean a return across sessions), and an *edit* be the number of changed lines between two consecutive submissions.

Table A.1: The derived thresholds.

Cut-off	Definition	Exam	Year
pass	score ≥ 50 (institutional: the course pass mark)	50	50
quick gap	gap $\leq P_{10}$ of gaps	≤ 12 s	≤ 11 s
long gap	gap $\geq P_{90}$ of gaps	≥ 243 s	≥ 273 s
small edit	edit $\leq P_{50}$ (median) of edit sizes	≤ 1 line	≤ 2 lines
large edit	edit $\geq P_{90}$ of edit sizes	≥ 6 lines	≥ 9 lines
big score gain	score rise ≥ 50 (half of the 100 marks)	50	50

P_p is the p-th percentile of the stated distribution, taken over all (student, question) pairs in that dataset.

Each feature is computed for one (student, question) first, then reduced to one value per student by the median over that student's questions. So, for example, the "quick resubmissions" feature of a student is the median over their questions of the share of that question's gaps that are quick.

Choosing the number of groups

The number of groups k is chosen from the data, not fixed in advance, using four standard measures across $k = 2$ to 8 (silhouette, subsample stability, Calinski-Harabasz, Davies-Bouldin; see Chapter 5 and Appendix B). The best-supported cut is $k = 4$ for the exam and $k = 3$ for the year. The full numeric tables are in Appendix B.

Appendix B

Statistical Tables

This appendix holds the numeric tables behind Chapter 5: how the number of groups k was chosen, and the full year↔exam correspondence at different year k (the robustness check).

Choosing the number of groups

Four internal measures across $k = 2$ to 8 (silhouette and Calinski-Harabasz: higher is better; Davies-Bouldin: lower is better; subsample stability: higher is better). The rank column aggregates the four (lower is better). The exam is best supported at $k = 4$ (Table B.1), the year at $k = 3$ (Table B.2).

Table B.1: Exam: cluster-quality measures by k (de-dup features, $n = 569$).

k	silhouette	stability	Calinski-H	Davies-B	rank
2	0.256	0.45	76	2.16	8
3	0.183	0.52	65	1.95	7
4	**0.185**	**0.54**	**61**	**1.82**	**4**
5	0.131	0.45	58	2.06	14

Table B.2: Year: cluster-quality measures by k (de-dup features, $n = 569$).

k	silhouette	stability	Calinski-H	Davies-B	rank
2	0.142	0.52	73	2.43	17
3	**0.164**	**0.60**	**75**	1.89	**5**
4	0.152	0.53	70	**1.81**	14
5	0.153	0.57	69	1.46	9

Year↔exam correspondence, robust to the year k

Chapter 5 reports the year↔exam link at year $k = 3$. Here we fix the exam at $k = 4$ and vary the year between $k = 2, 3, 4$. The association stays weak for every year k (Table B.3): the more groups we cut the year into, the more the big year group is split into thinner slices that still land in the same exam profile (E3). So the weak link is not an artefact of the choice of k.

Table B.3: Strength of the year↔exam link across the year k (exam fixed at $k = 4$).

year k	Cramér's V	adjusted Rand index	χ^2 p
2	0.213	0.044	< 0.001
3	0.159	0.074	< 0.001
4	0.134	0.033	< 0.001

Table B.4 gives $P(\text{exam} \mid \text{year})$, where each row sums to 100% and the base rate is $P(\text{exam})$:

Table B.4: $P(\text{exam} \mid \text{year})$ at year $k = 2, 3, 4$ (exam fixed at $k = 4$).

year k	group	E1	E2	E3	E4
2	Y1	5%	23%	64%	8%
	Y2	1%	11%	80%	8%
3	Y1	6%	23%	61%	10%
	Y2	3%	22%	69%	6%
	Y3	1%	11%	80%	8%
4	Y1	6%	23%	61%	10%
	Y2	3%	12%	80%	5%
	Y3	3%	22%	69%	6%
	Y4	0%	10%	81%	9%
base rate $P(\text{exam})$		3%	16%	73%	8%

$P(\text{year} \mid \text{exam})$, where each exam column sums to 100% within a given year k:

Table B.5: $P(\text{year} \mid \text{exam})$ at year $k = 2, 3, 4$ (exam fixed at $k = 4$).

year k	group	E1	E2	E3	E4
2	Y1	80%	62%	38%	45%
	Y2	20%	38%	62%	55%
3	Y1	60%	40%	23%	32%
	Y2	20%	22%	15%	13%
	Y3	20%	38%	62%	55%
4	Y1	60%	40%	23%	32%
	Y2	13%	10%	14%	9%
	Y3	20%	22%	15%	13%
	Y4	7%	28%	48%	47%

The four measures under the other linkage rules

Section 5.3 chooses Ward because the other rules take the students one at a time instead of dividing them. The same four measures used to choose k can be computed under each rule, which shows what those rules are doing and why the measures alone cannot decide between them. Tables B.6 and B.7 give the four values at the chosen k, beside the size of the largest group.

Table B.6: Exam, at $k = 4$: the four measures under each linkage rule, and the size of the largest group.

Linkage	silhouette	stability	Calinski-H	Davies-B	largest group
Ward	0.185	0.54	**61**	1.82	**417/569**
complete	0.599	0.77	24	0.49	563/569
average	0.599	0.80	24	0.49	563/569
single	**0.668**	0.77	18	**0.22**	566/569

Table B.7: Year, at $k = 3$: the four measures under each linkage rule, and the size of the largest group.

Linkage	silhouette	stability	Calinski-H	Davies-B	largest group
Ward	0.164	0.60	**75**	1.89	**321/569**
complete	0.684	0.57	27	0.20	567/569
average	0.684	0.74	27	0.20	567/569
single	0.684	**0.80**	27	**0.20**	567/569

How to read these tables. Three of the four measures are better for the other rules than for Ward, and on both datasets. The last column says why. Single

linkage puts 566 of the 569 exam students in one group and leaves three on their own. A grouping like that scores well on silhouette, because almost every student sits inside one large group and the few that do not are very far away, and it is stable under resampling for the same reason: take another 80% of the students and the same one big group comes back. It scores well on Davies-Bouldin because there is almost nothing to compare the big group against. What it does not do is divide the students, which is what this thesis needs. Only Calinski-Harabasz, which compares the spread between groups with the spread inside them, prefers Ward.

This is the clearest case in this work of a point made in Section 5.3: a measure of separation is not a verdict. Read alone, these numbers would recommend a grouping that describes nothing.

The same four measures are plotted across k for each rule in Figures B.1, B.2 and B.3, to be read against Figure 5.6, which shows Ward. Complete and average linkage behave alike. Single linkage is the one to look at: its curves are almost flat, because raising k takes one more student out of the large group instead of dividing it.

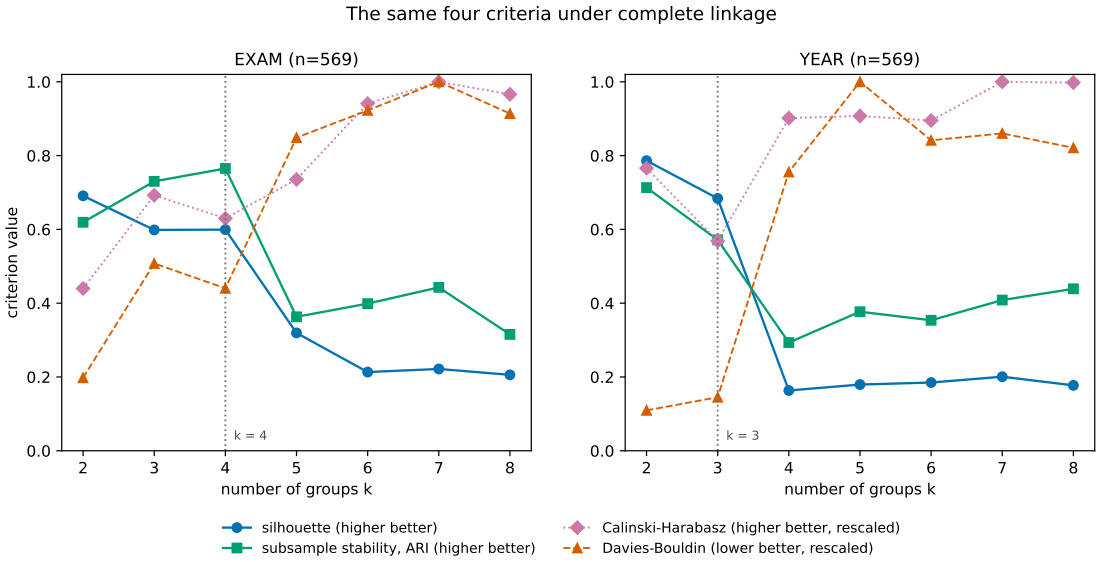

Figure B.1: Complete linkage: the same four measures across k, for the exam (left) and the year (right).

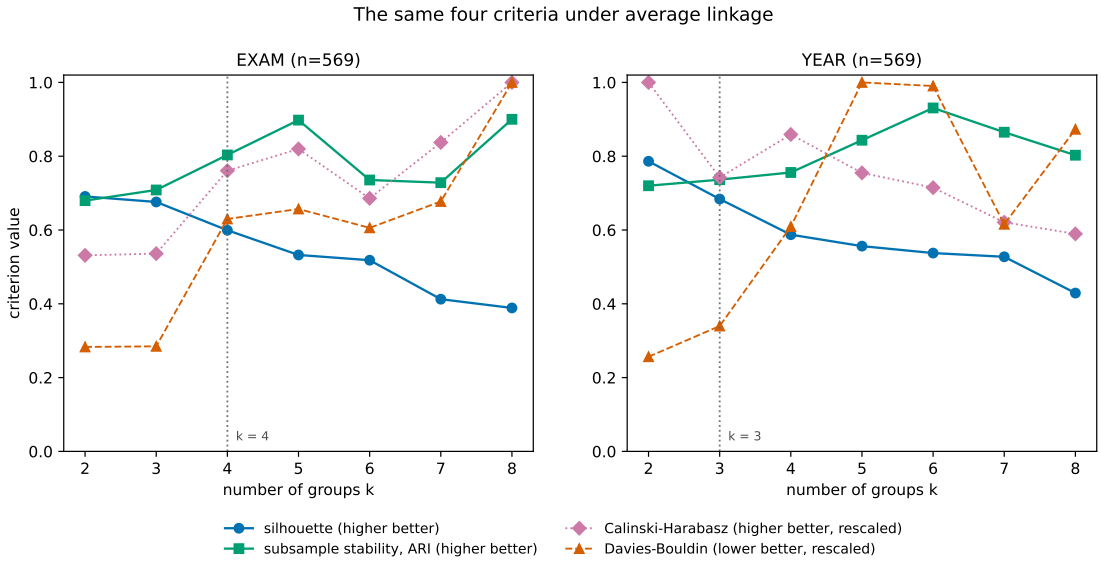

Figure B.2: Average linkage: the same four measures across k.

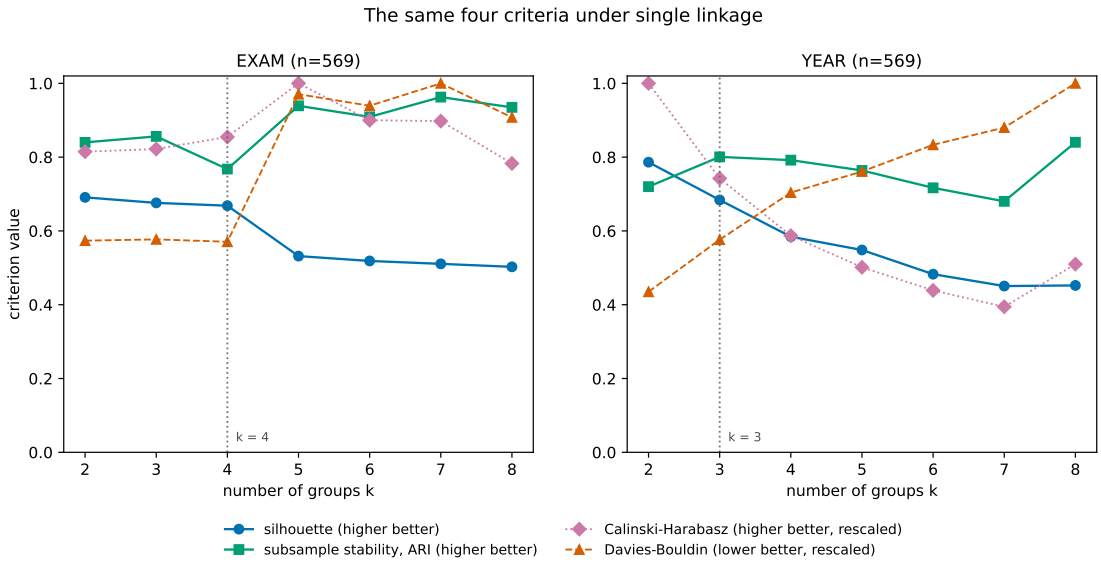

Figure B.3: Single linkage: the same four measures across k.

Appendix C

Additional case-study material

Figures and code for the case-study students referred to in Chapter 6 but not shown there in full: the E1 contrasting student, and group E2 in full (medoid and contrasting student). Each trajectory shows the score (left axis) and the edit size per submission (right axis), as in Chapter 6; code is transliterated to plain ASCII for typesetting (accents removed); the color marks the exam group.

E1 contrasting student

Listing C.1: E1 contrasting student (hash `855f65a4`), last of 3 submissions on q3 (score 0). A short, unfinished attempt; the student stopped after three tries.

```
def affecter_tables(tables, reservations):
    """
    Assigne des tables a des reservations selon les regles evoquees dans l'enonce.
    @pre: ``tables`` est une liste de tuples ``(numero_table, capacite)``
          ``reservations`` est une liste de tuples ``(nom, taille_groupe)``
    @post: retourne un dictionnaire ``{nom: numero_table ou None, ...}`` qui fait correspondre
     chaque
           reservation au numero de table affecte, ou ``None`` si aucune table disponible ne
     peut
           accueillir le groupe, selon les regles deterministes de l'enonce.
    """
    tables = [numero_table, capacite]
    numero_tables >= 1
    if capacite = 0:
        tables = 0
```

Listing C.2: E1 contrast, edit (submission 0 to 1). The student deletes most of a sketched attempt; the score stays at 0 and they stop after one more try. Removed lines in red, added lines in green.

```
            reservation au numero de table affecte, ou ``None`` si aucune table disponible ne
    peut
            accueillir le groupe, selon les regles deterministes de l'enonce.
-       """
-       table = [numero_table, capacite]
-       numero_tables >= 1
-       reservation >= 1
-       (nom, taille_groupe) =  reservation
-       reservation = [nom, taille_groupe]
-       taille_groupe >= 1
-       numero_table = numero_table or None
-       return {nom: numero_table ou None, ...}
+       """
```

E2 (grade $\approx$ 12)

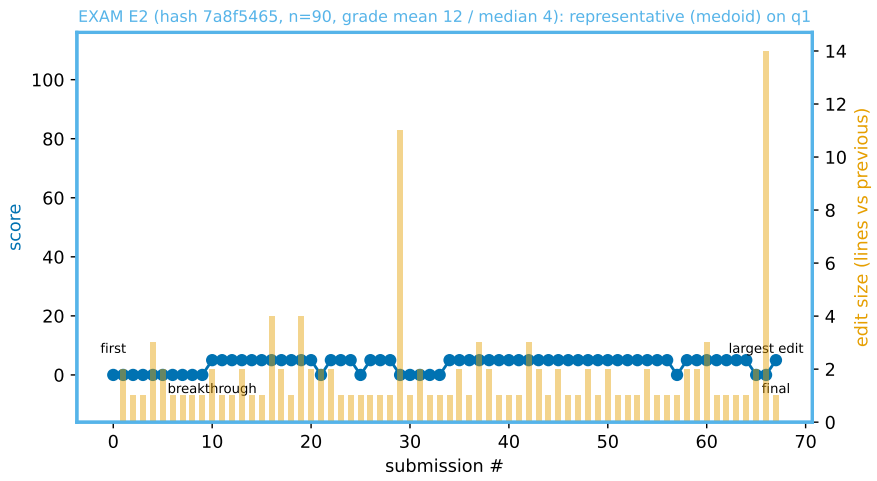

Figure C.1: E2 medoid: 68 submissions on q1, the score settling at a low partial value (5).

Listing C.3: E2 medoid (hash `7a8f5465`),last of 68 submissions on q1 (transpose), score 5. A transpose attempt that loops up to len(image) minus one and stores string-formatted entries; the score stayed at 5 through most of the sequence.

```
def transpose(image):
    f=0
    result=[]
    im=len(image)-1
    for f in range(im):
        colonne=[]
        for i in range(im):
            colonne.append(f"{str(image[i][f])}")
        result.append(colonne)

    return result
```

Listing C.4: E2 medoid, decisive edit (submission 28 to 29). The transpose loop is rewritten and breaks; the score falls from 5 to 0, and later returns to 5. Removed lines in red, added lines in green.

```
     """
     result=[]
-    nb=len(image)
-    f=0
-    point=0
-    while f<(nb+1):
-        for n in range(nb):
-            colonne=[]
-            for i in range(nb):
-                point=image[n][i]
-                colonne.append.point()
-            result.append.colonne()
+    for f<=len(image):
+        colonne=[]
+        for i in image:
+            colonne.append(image[i][f])
+        result.append(colonne)
         f+=1
-    return result
+    return colonne
```

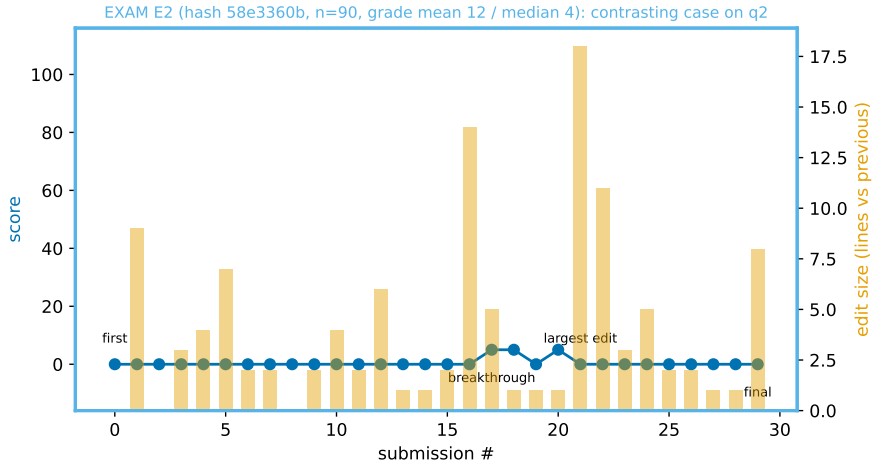

Figure C.2: E2 contrasting student: 30 submissions on q2, finishing at 0.

Listing C.5: E2 contrast (hash `58e3360b`), last of 30 submissions on q2 (score 0).

```
def conway(lst) :
    """
    Calcule le terme suivant de la suite de Conway a partir de ``lst``.
    @pre:  ``lst`` est une liste non-vide d'entiers positifs
    @post: retourne une liste representant le terme suivant ``lst``
    dans la suite de Conway. Pour chaque sequence de ``n`` nombres identiques ``x``
    consecutifs dans ``lst`` (n  1),  ``conway(lst)`` contient les nombres ``n`` et ``x``.

    Exemple: ``conway([1,1,1,2,2,1]) == [3,1,2,2,1,1]`` (trois fois 1, deux fois 2, une fois 1)
    """
    l=[]
```

Listing C.6: E2 contrast, decisive edit (submission 20 to 21). A docstring is added and the loop rewritten, but it breaks; the score falls from 5 to 0. Removed lines in red, added lines in green.

```
 def conway(lst) :
+    """
+    Calcule le terme suivant de la suite de Conway a partir de ``lst``.
+    @pre:  ``lst`` est une liste non-vide d'entiers positifs
+    @post: retourne une liste representant le terme suivant ``lst``
+    dans la suite de Conway. Pour chaque sequence de ``n`` nombres identiques ``x``
+    consecutifs dans ``lst`` (n  1),  ``conway(lst)`` contient les nombres ``n`` et ``x``.
+
+    Exemple: ``conway([1,1,1,2,2,1]) == [3,1,2,2,1,1]`` (trois fois 1, deux fois 2, une fois
     1)
+    """
+    count = 1
     l=[]
-    count = 1
-    i=0
```

CONTINUES ↓

↑ CONTINUED

```
-    while lst [i] == lst[i+1]:
-        count += 1
-        i += 1
-    l.append(count)
-    l.append(i)
-    count = 1
+    for i in range(len(lst)):
+        j = i +1
+        while lst[i] == lst[j]:
+            j+=1
+            count += 1
+        l.append[count]
+        l.append[i]
+        count = 1
     return l
```

Behavior across settings: the other case students

The same student across the year and the exam, for the seven case students not shown in Chapter 6 (Figure 6.11 shows the eighth, the E4 medoid). Each pair of bars is read as in that figure: the left bar is the student's year episodes, the right bar their exam episodes, split by shape; each title names the student's year and exam groups.

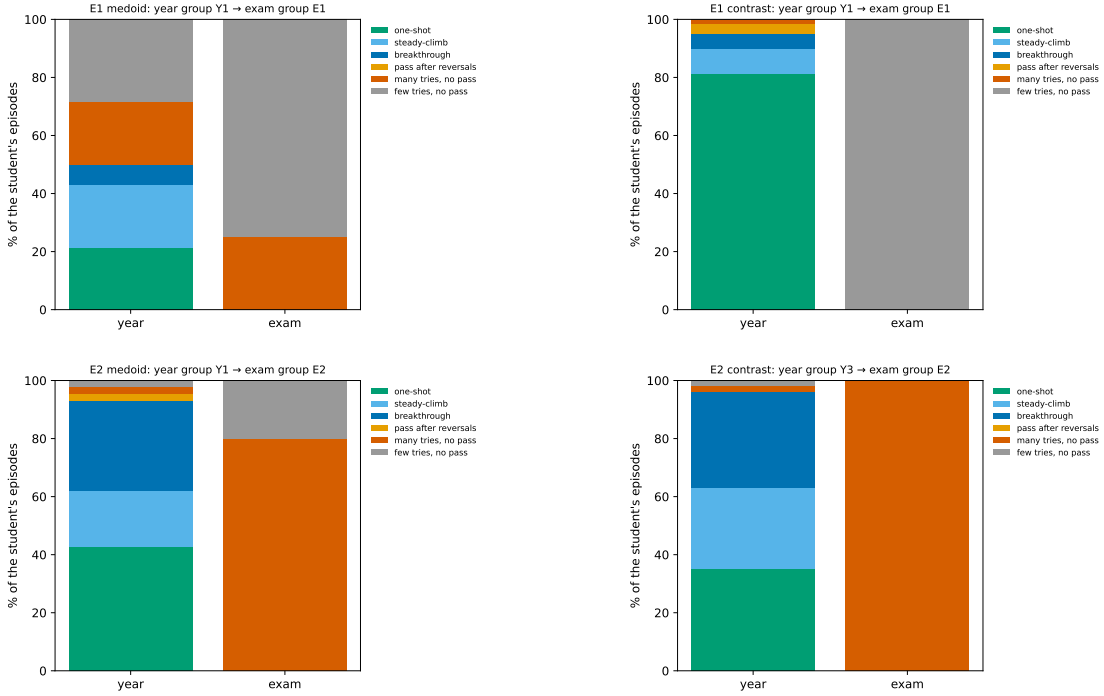

Figure C.3: Year-versus-exam episode-shape mix, groups E1 and E2 (medoid and contrasting student). Read as Figure 6.11; each title names the student's year and exam groups.

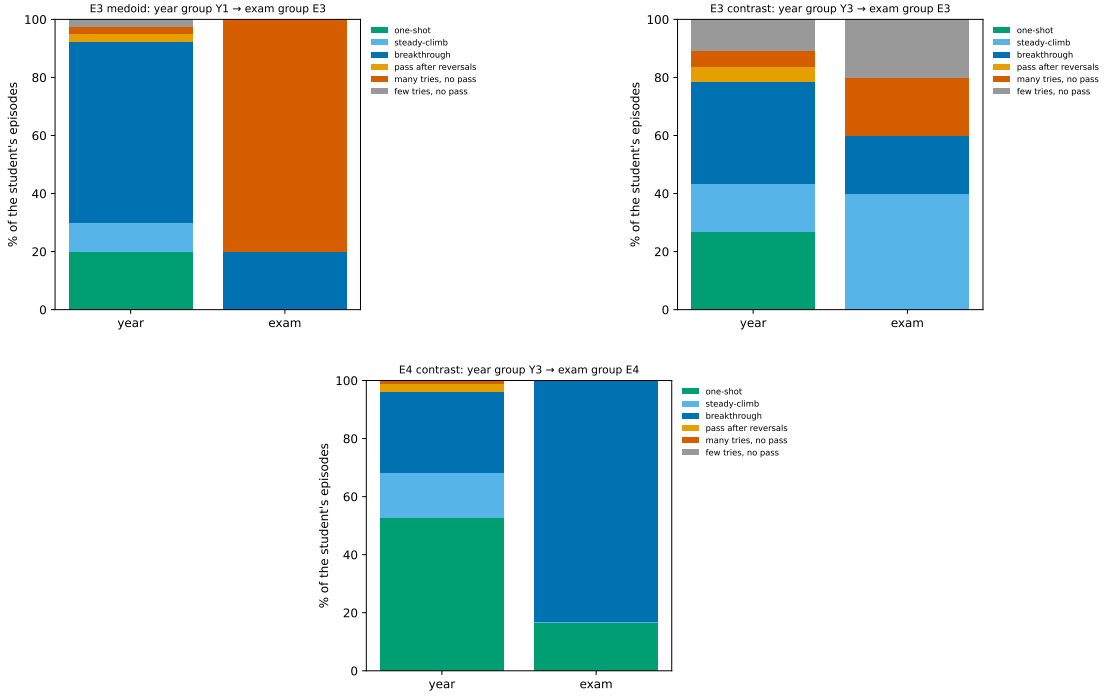

Figure C.4: Year-versus-exam episode-shape mix, group E3 (medoid and contrasting student) and the E4 contrasting student. The E4 medoid is in Figure 6.11.

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain
Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | **www.uclouvain.be/epl**